

图 5-14 根据 B 给出的窗口值，A 构造出自己的发送窗口

我们先讨论发送方 A 的发送窗口。发送窗口表示：在没有收到 B 的确认的情况下，A 可以连续把窗口内的数据都发送出去。凡是已经发送过的数据，在未收到确认之前都必须暂时保留，以便在超时重传时使用。

发送窗口里面的序号表示允许发送的序号。显然，窗口越大，发送方就可以在收到对方确认之前连续发送更多的数据，因而可能获得更高的传输效率。在上面的 5.5 节我们已经讲过，接收方会把自己的接收窗口数值放在窗口字段中发送给对方。因此，A 的发送窗口一定不能超过 B 的接收窗口数值。在后面的 5.8 节我们将要讨论，发送方的发送窗口大小还要受到当时网络拥塞程度的制约。但在目前，我们暂不考虑网络拥塞的影响。

发送窗口后沿的后面部分表示已发送且已收到了确认。这些数据显然不需要再保留了。而发送窗口前沿的前面部分表示不允许发送，因为接收方没有为这部分数据保留临时存放的缓存空间。

发送窗口的位置由窗口前沿和后沿的位置共同确定。发送窗口后沿的变化情况有两种可能，即不动（没有收到新的确认）和前移（收到了新的确认）。发送窗口后沿不可能向后移动，因为不能撤销掉已收到的确认。发送窗口前沿通常是不断向前移动的，但也有可能不动。这对应两种情况：一是没有收到新的确认，对方通知的窗口大小也不变；二是收到了新的确认但对方通知的窗口缩小了，使得发送窗口前沿正好不动。

发送窗口前沿也有可能向后收缩。这发生在对方通知的窗口缩小了。但 TCP 的标准强烈不赞成这样做。因为很可能发送方在收到这个通知以前已经发送了窗口中的许多数据，现在又要收缩窗口，不让发送这些数据，这样就会产生一些错误。

现在假定 A 发送了序号为 31 ~ 41 的数据。这时，发送窗口位置并未改变（如图 5-15 所示），但发送窗口内靠后面有 11 个字节（灰色方框表示）表示已发送但未收到确认。而发送窗口内靠前面的 9 个字节（序号 42 ~ 50）是允许发送但尚未发送的。

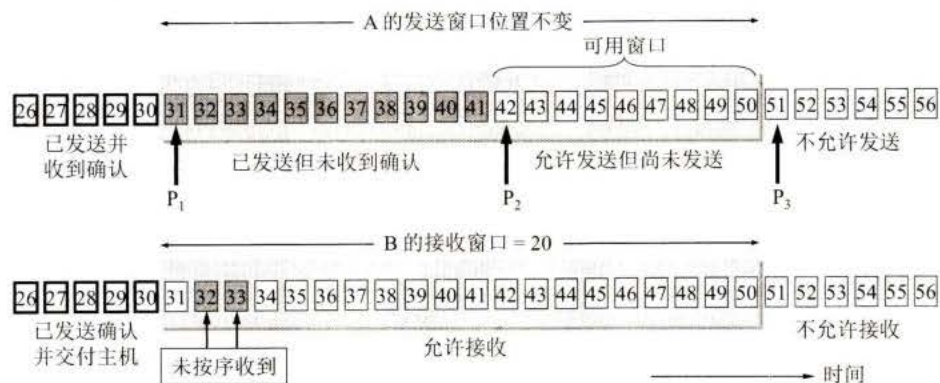


图 5-15 A 发送了 11 个字节的数据

从以上所述可以看出，要描述一个发送窗口的状态需要三个指针： P_1 、 P_2 和 P_3 （如图 5-15 所示）。指针都指向字节的序号。A 的发送窗口中三个指针指向的几个部分的意义如下：

P_1 之前的数据（序号 < 31）是已发送并已收到确认的部分。

P_3 之后的数据（序号 > 50）是不允许发送的部分。

$P_3 - P_1 = A$ 的发送窗口 = 20（序号 31 ~ 50）。

$P_2 - P_1 =$ 已发送但尚未收到确认的字节数（序号 31 ~ 41）。

$P_3 - P_2 =$ 允许发送但当前尚未发送的字节数（序号 42 ~ 50）（又称为可用窗口或有效窗口）。

再看一下 B 的接收窗口。设 B 的接收窗口大小是 20。在接收窗口外面，到序号为 30 的数据是已经发送过确认，并且已经交付主机了。因此在 B 可以不再保留这些数据。接收窗口内的数据（序号 31 ~ 50）是允许接收的。在图 5-15 中，B 收到了序号为 32 和 33 的数据，但序号为 31 的数据没有收到（也许丢失了，也许滞留在网络中的某处）。请注意，B 只能对按序收到的数据中的最高序号给出确认，因此 B 发送的确认报文段中的确认号仍然是 31（即期望收到的序号）。

现在假定 B 收到了序号为 31 的数据，把序号为 31 ~ 33 的数据交付主机，删除这些数据。接着把接收窗口向前移动 3 个序号（如图 5-16 所示），同时给 A 发送确认，其中窗口值仍为 20，但确认号是 34。这表明 B 已经收到了到序号 33 为止的数据。我们注意到，B 还收到了序号为 37、38 和 40 的数据，但这些数据都没有按序到达，只能先暂存在接收窗口中。A 收到 B 的确认后，就可以把发送窗口向前滑动 3 个序号，但指针 P_2 不动。可以看出，现在 A 的可用窗口增大了些，可发送的序号范围是 42 ~ 53。

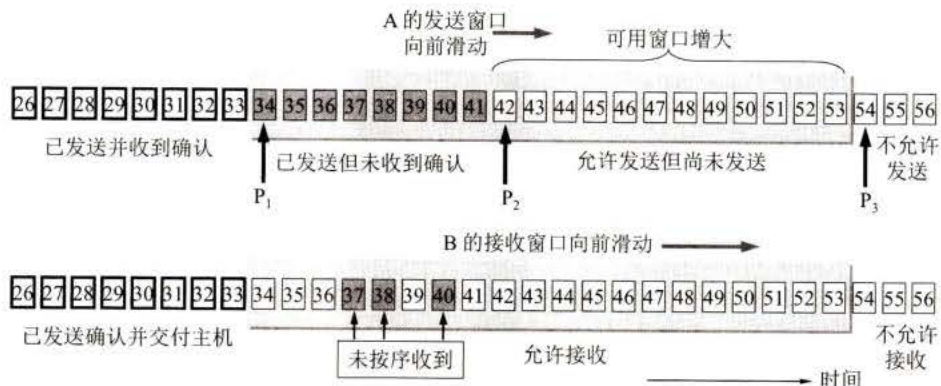


图 5-16 A 收到新的确认号，发送窗口向前滑动

A 在继续发送完序号 42 ~ 53 的数据后，指针 P_2 向前移动和 P_3 重合。发送窗口内的序号都已用完，但还没有再收到确认（如图 5-17 所示）。由于 A 的发送窗口已满，可用窗口已减小到零，因此必须停止发送。请注意，存在下面这种可能性，就是发送窗口内所有的数据都已正确到达 B，B 也早已发出了确认。但不幸的是，所有这些确认都滞留在网络中。在没有收到 B 的确认时，为了保证可靠传输，A 只能认为 B 还没有收到这些数据。于是，A 在经过一段时间后（由超时计时器控制）就重传这部分数据，重新设置超时计时器，直到收到 B 的确认为止。如果 A 按序收到落在发送窗口内的确认号，那么 A 就可以使发送窗口继续向前滑动，并发送新的数据。



图 5-17 A 的发送窗口内的序号都属于已发送但未被确认

我们在前面的图 5-7 中曾给出了这样的概念：发送方的应用进程把字节流写入 TCP 的发送缓存，接收方的应用进程从 TCP 的接收缓存中读取字节流。下面我们就进一步讨论前面讲的窗口和缓存的关系。图 5-18 画出了发送方维持的发送缓存和发送窗口，以及接收方维持的接收缓存和接收窗口。这里首先要明确两点：

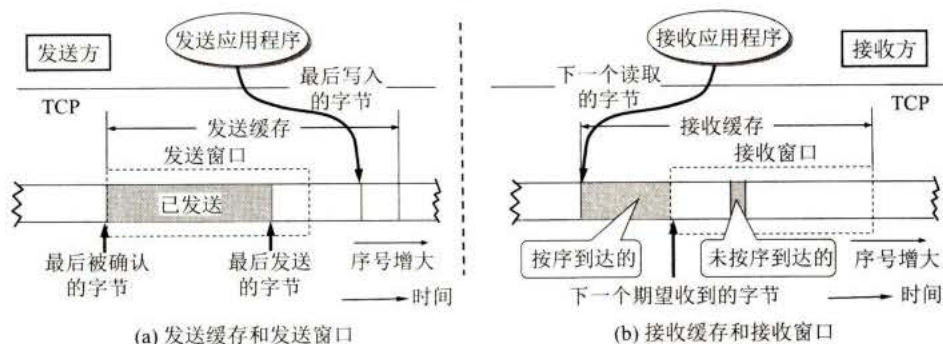


图 5-18 TCP 的缓存和窗口的关系

第一，缓存空间和序号空间都是有限的，并且都是循环使用的。最好是把它们画成圆环状的。但这里为了画图的方便，我们还是把它们画成了长条状的。

第二，由于缓存或窗口中实际的字节数可能很大，因此图 5-18 仅仅是个示意图，没有标出具体的数值。但用这样的图来说明缓存和发送窗口以及接收窗口的关系是很清楚的。

我们先看一下图 5-18(a)所示的发送方的情况。

发送缓存用来暂时存放：

- (1) 发送应用程序传送给发送方 TCP 准备发送的数据；
- (2) TCP 已发送出但尚未收到确认的数据。

发送窗口通常只是发送缓存的一部分。已被确认的数据应当从发送缓存中删除，因此发送缓存和发送窗口的后沿是重合的。发送应用程序最后写入发送缓存的字节减去最后被确认的字节，就是还保留在发送缓存中的被写入的字节数。发送应用程序必须控制写入缓存的速率，不能太快，否则发送缓存就会没有存放数据的空间。

再看一下图 5-18(b)所示的接收方的情况。

接收缓存用来暂时存放：

- (1) 按序到达的、但尚未被接收应用程序读取的数据；
- (2) 未按序到达的数据。

如果收到的分组被检测出有差错，则要丢弃。如果接收应用程序来不及读取收到的数据，接收缓存最终就会被填满，使接收窗口减小到零。反之，如果接收应用程序能够及时从接收缓存中读取收到的数据，接收窗口就可以增大，但最大不能超过接收缓存的大小。

图 5-18(b)中还指出了下一个期望收到的字节号。这个字节号也就是接收方给发送方的报文段的首部中的确认号。

根据以上所讨论的, 我们还要再强调以下三点。

第一, 虽然 A 的发送窗口是根据 B 的接收窗口设置的, 但在同一时刻, A 的发送窗口并不总是和 B 的接收窗口一样大。这是因为通过网络传送窗口值需要经历一定的时间滞后(这个时间是不确定的)。另外, 正如后面 5.7 节将要讲到的, 发送方 A 还可能根据网络当时的拥塞情况适当减小自己的发送窗口数值。

第二, 对于不按序到达的数据应如何处理, TCP 标准并无明确规定。如果接收方把不按序到达的数据一律丢弃, 那么接收窗口的管理将会比较简单, 但这样做对网络资源的利用不利(因为发送方会重复传送较多的数据)。因此 TCP 通常是把不按序到达的数据先临时存放在接收窗口中, 等到字节流中所缺少的字节收到后, 再按序交付上层的应用进程。

第三, TCP 要求接收方必须有累积确认的功能, 这样可以减小传输开销。接收方可以在合适的时候发送确认, 也可以在自己有数据要发送时把确认信息顺便捎带上。但请注意两点。一是接收方不应过分推迟发送确认, 否则会导致发送方不必要的重传, 这反而浪费了网络的资源。TCP 标准规定, 确认推迟的时间不应超过 0.5 秒。若收到一连串具有最大长度的报文段, 则必须每隔一个报文段就发送一个确认[RFC 1122, STD3]。二是捎带确认实际上并不经常发生, 因为大多数应用程序很少同时在两个方向上发送数据。

最后再强调一下, TCP 的通信是全双工通信。通信中的每一方都在发送和接收报文段。因此, 每一方都有自己的发送窗口和接收窗口。在谈到这些窗口时, 一定要弄清是哪一方的窗口。

5.6.2 超时重传时间的选择

上面已经讲到, TCP 的发送方在规定的时间内没有收到确认就要重传已发送的报文段。这种重传的概念是很简单的, 但重传时间的选择却是 TCP 最复杂的问题之一。

由于 TCP 的下层是互联网环境, 发送的报文段可能只经过一个高速率的局域网, 也可能经过多个低速率的网络, 并且每个 IP 数据报所选择的路由还可能不同。如果把超时重传时间设置得太短, 就会引起很多报文段的不必要的重传, 使网络负荷增大。但若把超时重传时间设置得过长, 则又使网络的空闲时间增大, 降低了传输效率。

那么, 运输层的超时计时器的超时重传时间究竟应设置为多大呢?

TCP 采用了一种自适应算法, 它记录一个报文段发出的时间, 以及收到相应的确认的时间。这两个时间之差就是报文段的往返时间 RTT。TCP 保留了 RTT 的一个加权平均往返时间 RTT_S (这又称为平滑的往返时间, S 表示 Smoothed。因为进行的是加权平均, 因此得出的结果更加平滑)。每当第一次测量到 RTT 样本时, RTT_S 值就取为所测量到的 RTT 样本值。但以后每测量到一个新的 RTT 样本, 就按下式重新计算一次 RTT_S :

$$\text{新的 } RTT_S = (1 - \alpha) \times (\text{旧的 } RTT_S) + \alpha \times (\text{新的 RTT 样本}) \quad (5-4)$$

在上式中, $0 \leq \alpha < 1$ 。若 α 很接近于零, 表示新的 RTT_S 值和旧的 RTT_S 值相比变化不大, 而对新的 RTT 样本影响不大(RTT 值更新较慢)。若选择 α 接近于 1, 则表示新的 RTT_S 值受新的 RTT 样本的影响较大(RTT 值更新较快)。已成为建议标准的 RFC 6298 推荐的 α 值为 1/8, 即 0.125。用这种方法得出的加权平均往返时间 RTT_S 就比测量出的 RTT 值

更加平滑。

显然，超时计时器设置的**超时重传时间 RTO (RetransmissionTime-Out)**应略大于上面得出的加权平均往返时间 RTT_S 。RFC 6298 建议使用下式计算 RTO:

$$RTO = RTT_S + 4 \times RTT_D \quad (5-5)$$

而 RTT_D 是 RTT 的**偏差**的加权平均值，它与 RTT_S 和新的 RTT 样本之差有关。RFC 6298 建议这样计算 RTT_D 。当第一次测量时， RTT_D 值取为测量到的 RTT 样本值的一半。在以后的测量中，则使用下式计算加权平均的 RTT_D :

$$\text{新的 } RTT_D = (1 - \beta) \times (\text{旧的 } RTT_D) + \beta \times |RTT_S - \text{新的 } RTT \text{ 样本}| \quad (5-6)$$

这里 β 是个小于 1 的系数，它的推荐值是 $1/4$ ，即 0.25。

上面所说的往返时间的测量，实现起来相当复杂。试看下面的例子。

如图 5-19 所示，发送出一个报文段，设定的重传时间到了，还没有收到确认，于是重传报文段。经过了一段时间后，收到了确认报文段。现在的问题是：**如何判定此确认报文段是对先发送的报文段的确认，还是对后来重传的报文段的确认？**由于重传的报文段和原来的报文段完全一样，因此源主机在收到确认后，就无法做出正确的判断，而正确的判断对确定加权平均 RTT_S 的值关系很大。

若收到的确认是对重传报文段的确认，但却被源主机当成是对原来的报文段的确认，则这样计算出的 RTT_S 和超时重传时间 RTO 就会偏大。若后面再发送的报文段又是经过重传后才收到确认报文段，则按此方法得出的超时重传时间 RTO 就越来越长。

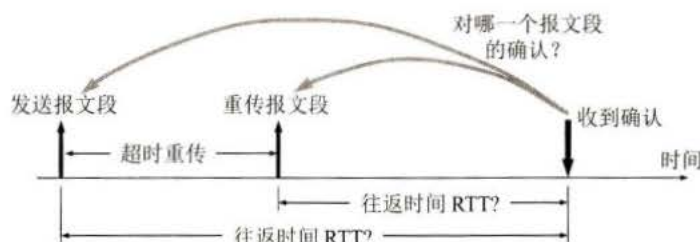


图 5-19 收到的确认是对哪一个报文段的确认？

同样，若收到的确认是对原来的报文段的确认，但被当成是对重传报文段的确认，则由此计算出的 RTT_S 和 RTO 都会偏小。这就必然导致报文段过多地重传。这样就有可能使 RTO 越来越短。

根据以上所述，Karn 提出了一个算法：在计算加权平均 RTT_S 时，只要报文段重传了，就不采用其往返时间样本。这样得出的加权平均 RTT_S 和 RTO 就较准确。

但是，这又引起新的问题。设想出现这样的情况：报文段的时延突然增大了很多。因此在原来得出的重传时间内不会收到确认报文段，于是就重传报文段。但根据 Karn 算法，不考虑重传的报文段的往返时间样本。这样，超时重传时间就无法更新。

因此要对 Karn 算法进行修正。方法是：报文段每重传一次，就把超时重传时间 RTO 增大一些。典型的做法是取新的重传时间为旧的重传时间的 2 倍。当不再发生报文段的重传时，才根据上面给出的式(5-5)计算超时重传时间。实践证明，这种策略较为合理。

总之，Karn 算法能够使运输层区分开有效的和无效的往返时间样本，从而改进了往返时间的估测，使计算结果更加合理。

5.6.3 选择确认 SACK

现在还有一个问题没有讨论。这就是若收到的报文段无差错，只是未按序号，中间还缺少一些序号的数据，那么能否设法只传送缺少的数据而不重传已经正确到达接收方的数据？答案是可行的。选择确认(Selective ACK)[RFC 2018，建议标准]就是一种可行的处理方法。

我们用一个例子来说明选择确认的工作原理。TCP 的接收方在接收对方发送过来的数据字节流的序号不连续，结果就形成了一些不连续的字节块（如图 5-20 所示）。可以看出，序号 1 ~ 1000 收到了，但序号 1001 ~ 1500 没有收到。接下来的字节流又收到了，可是又缺少了 3001 ~ 3500。再后面从序号 4501 起又没有收到。也就是说，接收方收到了和前面的字节流不连续的两个字节块。如果这些字节的序号都在接收窗口之内，那么接收方就先收下这些数据，但要把这些信息准确地告诉发送方，使发送方不要再重复发送这些已收到的数据。

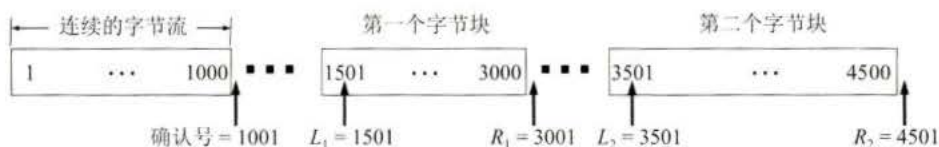


图 5-20 接收到的字节流序号不连续

从图 5-20 可看出，和前后字节不连续的每一个字节块都有两个边界：左边界和右边界，因此在图中用四个指针标记这些边界。请注意，第一个字节块的左边界 $L_1 = 1501$ ，但右边界 $R_1 = 3001$ 而不是 3000。这就是说，左边界指出字节块的第一个字节的序号，但右边界减 1 才是字节块的最后一个序号。同理，第二个字节块的左边界 $L_2 = 3501$ ，而右边界 $R_2 = 4501$ 。

我们知道，TCP 的首部没有哪个字段能够提供上述这些字节块的边界信息。RFC 2018 规定，如果要使用选择确认 SACK，那么在建立 TCP 连接时，就要在 TCP 首部的选项中加上“允许 SACK”的选项，而双方必须都事先商定好。如果使用选择确认，那么原来首部中的“确认号字段”的用法仍然不变。只是以后在 TCP 报文段的首部中都增加了 SACK 选项，以便报告收到的不连续的字节块的边界。由于首部选项的长度最多只有 40 字节，而指明一个边界就要用掉 4 字节（因为序号有 32 位，需要使用 4 个字节表示），因此在选项中最多只能指明 4 个字节块的边界信息。这是因为 4 个字节块共有 8 个边界，因而需要用 32 个字节来描述。另外还需要两个字节，一个字节用来指明是 SACK 选项，另一个字节指明这个选项要占用多少字节。如果要报告 5 个字节块的边界信息，那么至少需要 42 个字节。这就超过了选项长度 40 字节的上限。互联网建议标准 RFC 2018 还对报告这些边界信息的格式都做出了非常明确的规定，这里从略。

然而，SACK 文档并没有指明发送方应当怎样响应 SACK。因此大多数的实现还是重传所有未被确认的数据块。



5.7 TCP 的流量控制

5.7.1 利用滑动窗口实现流量控制

一般说来，我们总是希望数据传输得更快一些。但如果发送方把数据发送得过快，接收方就可能来不及接收，这就会造成数据的丢失。所谓**流量控制(flow control)**就是让发送方的发送速率不要太快，要让接收方来得及接收。

利用滑动窗口机制可以很方便地在 TCP 连接上实现对发送方的流量控制。

下面通过图 5-21 的例子说明如何利用滑动窗口机制进行流量控制。

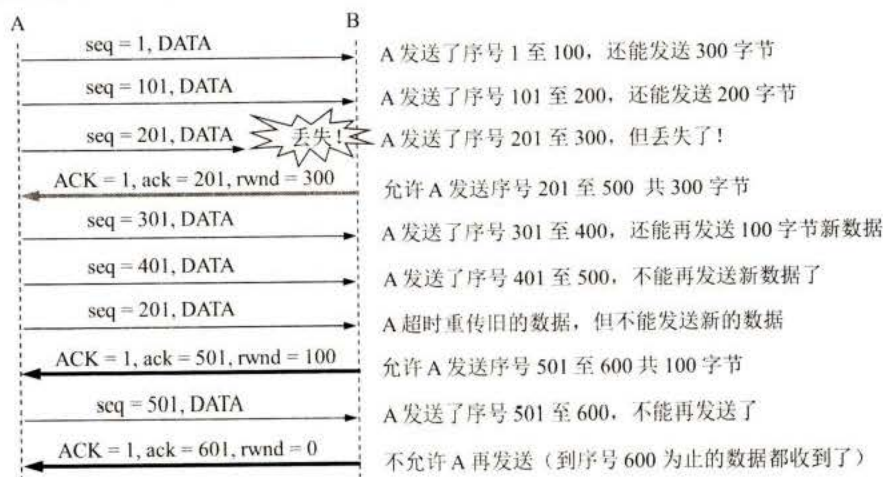


图 5-21 利用可变窗口进行流量控制举例

设 A 向 B 发送数据。在连接建立时，B 告诉了 A：“我的接收窗口 $rwnd = 400$ 。”（这里 $rwnd$ 表示 receiver window。）因此，发送方的发送窗口不能超过接收方给出的接收窗口^①的数值。请注意，TCP 的窗口单位是字节，不是报文段。TCP 连接建立时的窗口协商过程在图中没有显示出来。再设每一个报文段为 100 字节长，而数据报文段序号的初始值设为 1（见图中第一个箭头上面的序号 $seq = 1$ 。图中右边的注释可帮助理解整个过程）。请注意，图中箭头上上面大写 ACK 表示首部中的确认位 ACK，小写 ack 表示确认字段的值。

我们应注意到，接收方的主机 B 进行了三次流量控制。第一次把窗口减小到 $rwnd = 300$ ，第二次又减到 $rwnd = 100$ ，最后减到 $rwnd = 0$ ，即不允许发送方再发送数据了。这种使发送方暂停发送的状态将持续到主机 B 重新发出一个新的窗口值为止。我们还应注意到，B 向 A 发送的三个报文段都设置了 $ACK = 1$ ，只有在 $ACK = 1$ 时确认号字段才有意义。

现在我们考虑一种情况。在图 5-21 中，B 向 A 发送了零窗口的报文段后不久，B 的接收缓存又有了一些存储空间。于是 B 向 A 发送了 $rwnd = 400$ 的报文段。然而这个报文段在传送过程中丢失了。A 一直等待收到 B 发送的非零窗口的通知，而 B 也一直等待 A 发送的数据。如果没有其他措施，这种互相等待的死锁局面将一直延续下去。

为了解决这个问题，TCP 为每一个连接设有一个持续计时器(persistence timer)。只要

^① 注：从 $rwnd$ 的原文看，中文译名应当是接收方窗口。然而在不产生误解的情况下，也可简称为接收窗口。

TCP 连接的一方收到对方的零窗口通知，就启动持续计时器。若持续计时器设置的时间到期，就发送一个零窗口探测报文段（仅携带 1 字节的数据）^①，而对方就在确认这个探测报文段时给出了现在的窗口值。如果窗口仍然是零，那么收到这个报文段的一方就重新设置持续计时器。如果窗口不是零，那么死锁的僵局就可以打破了。

5.7.2 TCP 的传输效率

前面已经讲过，应用进程把数据传送到 TCP 的发送缓存后，剩下的发送任务就由 TCP 来控制了。可以用不同的机制来控制 TCP 报文段的发送时机。例如，第一种机制是 TCP 维持一个变量，它等于最大报文段长度 MSS。只要缓存中存放的数据达到 MSS 字节时，就组装成一个 TCP 报文段发送出去。第二种机制是由发送方的应用进程指明要求发送报文段，即 TCP 支持的推送(push)操作。第三种机制是发送方的一个计时器期限到了，这时就把当前已有的缓存数据装入报文段（但长度不能超过 MSS）发送出去。

但是，如何控制 TCP 发送报文段的时机仍然是一个较为复杂的问题[RFC 1122]。

例如，一个交互式用户使用一条 TELNET 连接（运输层为 TCP 协议）。假设用户只发 1 个字符，加上 20 字节的首部后，得到 21 字节长的 TCP 报文段。再加上 20 字节的 IP 首部，形成 41 字节长的 IP 数据报。在接收方 TCP 立即发出确认，构成的数据报是 40 字节长（假定没有数据发送）。若用户要求远地主机回送这一字符，则又要发回 41 字节长的 IP 数据报和 40 字节长的确认 IP 数据报。这样，用户仅发 1 个字符时，线路上就需传送总长度为 162 字节共 4 个报文段。当线路带宽并不富裕时，这种传送方法的效率的确不高。因此应当推迟发回确认报文，并尽量使用捎带确认的方法。

在 TCP 的实现中广泛使用 Nagle 算法。算法如下：若发送应用进程把要发送的数据逐个字节地送到 TCP 的发送缓存，则发送方就把第一个数据字节先发送出去，把后面到达的数据字节都缓存起来。当发送方收到对第一个数据字符的确认后，再把发送缓存中的所有数据组装成一个报文段发送出去，同时继续对随后到达的数据进行缓存。只有在收到对前一个报文段的确认后才继续发送下一个报文段。当数据到达较快而网络速率较慢时，用这样的方法可明显地减少所用的网络带宽。Nagle 算法还规定，当到达的数据已达到发送窗口大小的一半或已达到报文段的最大长度时，就立即发送一个报文段。这样做，就可以有效地提高网络的吞吐量。

另一个问题叫作糊涂窗口综合征(silly window syndrome)，有时也会使 TCP 的性能变坏。设想一种情况：TCP 接收方的缓存已满，而交互式的应用进程一次只从接收缓存中读取 1 个字节（这样就使接收缓存空间仅腾出 1 个字节），然后向发送方发送确认，并把窗口设置为 1 个字节（但发送的数据报是 40 字节长）。接着，发送方又发来 1 个字节的数据（请注意，发送方发送的 IP 数据报是 41 字节长）。接收方发回确认，仍然将窗口设置为 1 个字节。这样进行下去，使网络的效率很低。

要解决这个问题，可以让接收方等待一段时间，使得或者接收缓存已有足够空间容纳一个最长的报文段，或者等到接收缓存已有一半空闲的空间。只要出现这两种情况之一，接



^① 注：TCP 规定，即使设置为零窗口，也必须接收以下几种报文段：零窗口探测报文段、确认报文段和携带紧急数据的报文段。

收方就发出确认报文，并向发送方通知当前的窗口大小。此外，发送方也不要发送太小的报文段，而是把数据积累成足够大的报文段，或达到接收方缓存的空间的一半大小。

上述两种方法可配合使用。使得在发送方不发送很小的报文段的同时，接收方也不要再在缓存刚刚有了一点小的空间就急忙把这个很小的窗口大小信息通知给发送方。

5.8 TCP 的拥塞控制

5.8.1 拥塞控制的一般原理

在计算机网络中的链路容量（即带宽）、交换节点中的缓存和处理机等，都是网络的资源。在某段时间，若对网络中某一资源的需求超过了该资源所能提供的可用部分，网络的性能就要变坏。这种情况就叫作**拥塞**(congestion)。可以把出现网络拥塞的条件写成如下的关系式：

$$\sum \text{对资源的需求} > \text{可用资源} \quad (5-7)$$

若网络中有许多资源同时呈现供应不足，网络的性能就要明显变坏，整个网络的吞吐量将随输入负荷的增大而下降。

有人可能会说：“只要任意增加一些资源，例如，把节点缓存的存储空间扩大，或把链路更换为更高速率的链路，或把节点处理机的运算速度提高，就可以解决网络拥塞的问题。”其实不然。这是因为网络拥塞是一个非常复杂的问题。简单地采用上述做法，在许多情况下，不但不能解决拥塞问题，而且还可能使网络的性能更坏。

网络拥塞往往是由很多因素引起的。例如，当某个节点缓存的容量太小时，到达该节点的分组因无存储空间暂存而不得被丢弃。现在设想将该节点缓存的容量扩展到非常大，于是凡到达该节点的分组均可在节点的缓存队列中排队，不受任何限制。由于输出链路的容量和处理机的处理速度并未提高，因此在这队列中的绝大多数分组的排队等待时间将会大大增加，结果上层软件只好把它们进行重传（因为早就超时了）。由此可见，简单地扩大缓存的存储空间同样会造成网络资源的严重浪费，因而解决不了网络拥塞的问题。

又如，处理机处理的速率太低可能引起网络的拥塞。简单地将处理机的速率提高，可能会使上述情况缓解一些，但往往又会将瓶颈转移到其他地方。问题的实质往往是整个系统的各个部分不匹配。只有所有的部分都平衡了，问题才会得到解决。

拥塞常常趋于恶化。如果一个路由器没有足够的缓存空间，它就会丢弃一些新到的分组。但当分组被丢弃时，发送这一分组的源点就会重传这一分组，甚至可能还要重传多次。这样会引起更多的分组流入网络和被网络中的路由器丢弃。可见拥塞引起的重传并不会缓解网络的拥塞，反而会加剧网络的拥塞。

拥塞控制与流量控制的关系密切，它们之间也存在着一些差别。所谓**拥塞控制**就是防止过多的数据注入到网络中，这样可以使网络中的路由器或链路不至于过载。拥塞控制所要做的都有一个前提，就是网络能够承受现有的网络负荷。拥塞控制是一个全局性的过程，涉及所有的主机、所有的路由器，以及与降低网络传输性能有关的所有因素。但 TCP 连接的端点只要迟迟不能收到对方的确认信息，就猜想当前网络中的某处很可能发生了拥塞，但这时却无法知道拥塞到底发生在网络的何处，也无法知道发生拥塞的具体原因。（是访问某个服务器的通信量过大？还是在某个地区出现自然灾害？）

相反，**流量控制**往往是指点对点通信量的控制，是个端到端的问题（接收端控制发送

扫一扫



视频讲解

端)。流量控制所要做的就是抑制发送端发送数据的速率,以便接收端来得及接收。

可以用一个简单例子说明这种区别。设某个光纤网络的链路传输速率为 1000 Gbit/s,有一台巨型计算机向一台个人电脑以 1 Gbit/s 的速率传送文件。显然,网络本身的带宽是足够大的,因而不存在产生拥塞的问题。但流量控制却是必需的,因为巨型计算机必须经常停下来,以便个人电脑来得及接收。

但如果有另一个网络,其链路传输速率为 1 Mbit/s,而有 1000 台大型计算机连接在这个网络上。假定其中的 500 台计算机分别向其余的 500 台计算机以 100 kbit/s 的速率发送文件。那么现在的问题已不是接收端的大型计算机是否来得及接收,而是整个网络的输入负载是否超过网络所能承受的。

拥塞控制和流量控制之所以常常被弄混,是因为某些拥塞控制算法是向发送端发送控制报文,并告诉发送端,网络已出现麻烦,必须放慢发送速率。这点又和流量控制是很相似的。

流量控制和拥塞控制的区别可以用图 5-22 的简单比喻来说明。图中表示一水龙头通过管道向一个水桶放水。图 5-22(a)表示水桶太小,来不及接收注入水桶的水。这时只好请求管水龙头的人把水龙头拧小些,以减缓放水的速率。这就相当于流量控制。图 5-22(b)表示虽然水桶足够大,但管道中有很狭窄的地方,使得管道不通畅,水流被堵塞。这种情况被反馈到管水龙头的人,请求把水龙头拧小些,以减缓放水的速率,为的是减缓水管的堵塞状态。这就相当于拥塞控制。请注意,同样是把水龙头拧小些,但目的是很不一样的。



图 5-22 流量控制和拥塞控制的比喻(本图取自[TANE11]图 6-22,特此致谢)

进行拥塞控制需要付出代价。这首先需要获得网络内部流量分布的信息。在实施拥塞控制时,还需要在节点之间交换信息和各种命令,以便选择控制的策略和实施控制。这样就产生了额外开销。拥塞控制有时需要将一些资源(如缓存、带宽等)分配给个别用户(或一些类别的用户)单独使用,这样就使得网络资源不能更好地实现共享。十分明显,在设计拥塞控制策略时,必须全面衡量得失。

在图 5-23 中的横坐标是提供的负载(offered load),代表单位时间内输入给网络的分组数目。因此提供的负载也称为输入负载或网络负载。纵坐标是吞吐量(throughput),代表单位时间内从网络输出的分组数目。具有理想拥塞控制的网络,在吞吐量饱和之前,网络吞吐量应等于提供的负载,故吞吐量曲线是 45°的斜线。但当提供的负载超过某一限度时,由于网

络资源受限, 吞吐量不再增长而保持为水平线, 即吞吐量达到饱和。这就表明提供的负载中有一部分损失掉了 (例如, 输入到网络的某些分组被某个节点丢弃了)。虽然如此, 在这种理想的拥塞控制作用下, 网络的吞吐量仍然维持在其所能达到的最大值。

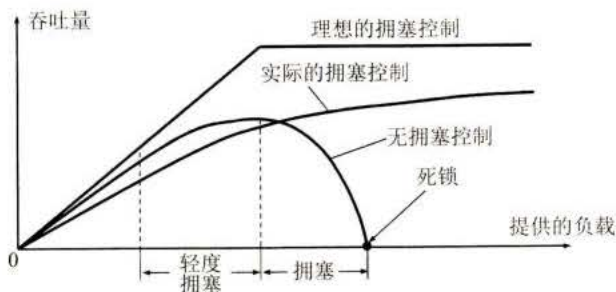


图 5-23 拥塞控制所起的作用

但是, 实际网络的情况就很不相同了。从图 5-23 可看出, 随着提供的负载的增大, 网络吞吐量的增长速率逐渐减小。也就是说, 在网络吞吐量还未达到饱和时, 就已经有一部分输入分组被丢弃了。当网络的吞吐量明显地小于理想的吞吐量时, 网络就进入了**轻度拥塞**的状态。更值得注意的是, 当提供的负载达到某一数值时, 网络的吞吐量反而随提供的负载的增大而下降, 这时**网络就进入了拥塞状态**。当提供的负载继续增大到某一数值时, 网络的吞吐量就下降到零, 网络已无法工作, 这就是所谓的**死锁(deadlock)**。

从原理上讲, 寻找拥塞控制的方案无非是寻找使不等式(5-7)不再成立的条件。这或者是增大网络的某些可用资源 (如业务繁忙时增加一些链路, 增大链路的带宽, 或使额外的通信量从另外的通路分流), 或减少一些用户对某些资源的需求 (如拒绝接受新的建立连接的请求, 或要求用户减轻其负荷, 这属于降低服务质量)。但正如上面所讲过的, 在采用某种措施时, 还必须考虑到该措施所带来的其他影响。

实践证明, 拥塞控制是很难设计的, 因为它是一个**动态的** (而不是静态的) 问题。当前网络正朝着高速化的方向发展, 这很容易出现缓存不够大而导致分组的丢失。但分组的丢失是网络发生拥塞的征兆而不是原因。在许多情况下, 甚至正是拥塞控制机制本身成为引起网络性能恶化甚至发生死锁的原因。**这点应特别引起重视。**

由于计算机网络是一个很复杂的系统, 因此可以从控制理论的角度来看拥塞控制这个问题。这样, 从大的方面看, 可以分为**开环控制**和**闭环控制**两种方法。开环控制就是在设计网络时事先将发生拥塞的有关因素考虑周到, 力求网络在工作时不产生拥塞。但一旦整个系统运行起来, 就不再中途进行改正了。

闭环控制是基于反馈环路的概念, 主要有以下几种措施:

- (1) 监测网络系统以便检测到拥塞在何时、何处发生。
- (2) 把拥塞发生的信息传送到可采取行动的地方。
- (3) 调整网络系统的运行以解决出现的问题。

有很多的方法可用来监测网络的拥塞。主要的一些指标是: 由于缺少缓存空间而被丢弃的分组的百分数、平均队列长度、超时重传的分组数、平均分组时延、分组时延的标准差, 等等。上述这些指标的上升都标志着拥塞发生的可能性增加。

一般在监测到拥塞发生时, 要将拥塞发生的信息传送到产生分组的源站。当然, 通知拥塞发生的分组同样会使网络更加拥塞。

另一种方法是在路由器转发的分组中保留一个比特或字段，用该比特或字段的值表示网络没有拥塞或产生了拥塞。也可以由一些主机或路由器周期性地发出探测分组，以询问拥塞是否发生。

此外，过于频繁地采取行动以缓和网络的拥塞，会使系统产生不稳定的振荡。但过于迟缓地采取行动又不具有任何实用价值。因此，要采用某种折中的方法，但选择正确的时间常数是相当困难的。

下面就来介绍更加具体的防止网络拥塞的方法。

5.8.2 TCP 的拥塞控制方法

TCP 进行拥塞控制的算法有四种，即慢开始(slow-start)、拥塞避免(congestion avoidance)、快重传(fast retransmit)和快恢复(fast recovery)（见草案标准 RFC 5681）。下面就介绍这些算法的原理。为了集中精力讨论拥塞控制，我们假定：

- (1) 数据是单方向传送的，对方只传送确认报文。
- (2) 接收方总是有足够大的缓存空间，因而发送窗口的大小由网络的拥塞程度来决定。

1. 慢开始和拥塞避免

下面讨论的拥塞控制也叫作**基于窗口的拥塞控制**。为此，发送方维持一个叫作**拥塞窗口** `cwnd` (congestion window)的状态变量。拥塞窗口的大小取决于网络的拥塞程度，并且是动态变化着的。发送方让自己的发送窗口等于拥塞窗口。根据假定，对方的接收窗口足够大，发送方在发送数据时，只需考虑发送方的拥塞窗口。

发送方控制拥塞窗口的原则是：只要网络没有出现拥塞，拥塞窗口就可以再增大一些，以便把更多的分组发送出去，这样就可以提高网络的利用率。但只要网络出现拥塞或有可能出现拥塞，就必须把拥塞窗口减小一些，以减少注入到网络中的分组数，以便缓解网络出现的拥塞。

发送方又如何知道网络发生了拥塞呢？我们知道，当网络发生拥塞时，路由器就要把来不及处理而排不上队的分组丢弃。因此只要发送方没有按时收到对方的确认报文，也就是说，只要出现了超时，就可以估计可能在网络某处出现了拥塞。现在通信线路的传输质量一般都很好，因传输出差错而丢弃分组的概率是很小的（远小于 1 %）。因此，发送方在超时重传计时器启动时，就**判断网络出现了拥塞**。

下面将讨论拥塞窗口 `cwnd` 的大小是怎样变化的。我们从“慢开始算法”讲起。

慢开始算法的思路是这样的：当主机在已建立的 TCP 连接上开始发送数据时，并不清楚网络当前的负荷情况。如果立即把大量数据字节注入到网络，那么就有可能引起网络发生拥塞。经验证明，较好的方法是先探测一下，即由小到大逐渐增大注入到网络中的数据字节，也就是说，由小到大逐渐增大拥塞窗口数值。

旧的规定是这样的：在刚刚开始发送报文段时，先把初始拥塞窗口 `cwnd` 设置为 1 至 2 个发送方的最大报文段 `SMSS` (Sender Maximum Segment Size)的数值，但新的 RFC 5681（草案标准）把初始拥塞窗口 `cwnd` 设置为不超过 2 至 4 个 `SMSS` 的数值。具体的规定如下：

若 $SMSS > 2190$ 字节，

则设置初始拥塞窗口 $cwnd = 2 \times SMSS$ 字节，且不得超过 2 个报文段。

若 $(SMSS > 1095 \text{ 字节})$ 且 $(SMSS \leq 2190 \text{ 字节})$ ，

扫一扫



视频讲解

则设置初始拥塞窗口 $cwnd = 3 \times SMSS$ 字节，且不得超过 3 个报文段。

若 $SMSS \leq 1095$ 字节，

则设置初始拥塞窗口 $cwnd = 4 \times SMSS$ 字节，且不得超过 4 个报文段。

可见这个规定就是限制初始拥塞窗口的字节数。

慢开始规定，在每收到一个对新的报文段的确认后，可以把拥塞窗口增加最多一个 $SMSS$ 的数值。更具体些，就是

$$\text{拥塞窗口 } cwnd \text{ 每次的增加量} = \min(N, SMSS) \quad (5-8)$$

其中 N 是原先未被确认的、但现在被刚收到的确认报文段所确认的字节数。不难看出，当 $N < SMSS$ 时，拥塞窗口每次的增加量要小于 $SMSS$ 。

用这样的方法逐步增大发送方的拥塞窗口 $cwnd$ ，可以使分组注入到网络的速率更加合理。

下面用例子说明慢开始算法的原理。请注意，虽然实际上 TCP 用字节数作为窗口大小的单位。但为叙述方便起见，我们用报文段的个数作为窗口大小的单位，这样可以使用较小的数字来阐明拥塞控制的原理。

在一开始发送方先设置 $cwnd = 1$ ，发送第一个报文段，接收方收到后就发送确认。慢开始算法规定，发送方每收到一个对新报文段的确认（对重传的确认不算在内），就把发送方的拥塞窗口加 1。因此，经过一个往返时延 RTT 后，发送方就增大拥塞窗口，使 $cwnd = 2$ ，即发送方现在可连续发送两个报文段。接收方收到这两个报文段后，先后发回两个确认。现在发送方收到两个确认，根据慢开始算法，拥塞窗口就应当加 2，使拥塞窗口从 $cwnd = 2$ 增加到 $cwnd = 4$ ，即可连续发送 4 个报文段。发送方收到这 4 个确认后，就可以把拥塞窗口再加 4，使 $cwnd = 8$ （如图 5-24 所示）。显然，发送方并不是要在所有的确认都收齐了之后才调整其拥塞窗口，而是收到一个确认就调整一下拥塞窗口，抓紧时间发送报文段。但这样的细节不是我们现在所要研究的，我们想知道的只是拥塞窗口的大致增长趋势。

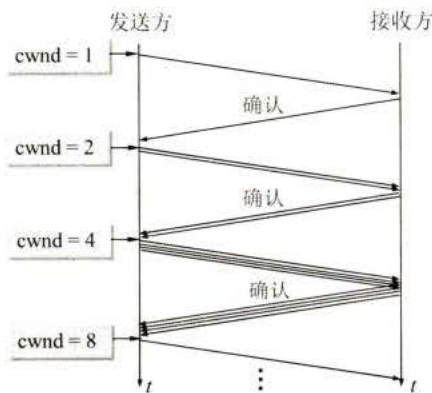


图 5-24 发送方每收到 1 个确认就把拥塞窗口加 1

由此可见，慢开始的“慢”并不是指 $cwnd$ 的增长速率慢，而是指在 TCP 开始发送报文段时，只发送一个报文段，即设置 $cwnd = 1$ ，目的是试探一下网络的拥塞情况，然后视情况再逐渐增大 $cwnd$ 。这当然比一开始设置大的 $cwnd$ 值，一下子把许多报文段迅速注入到网络要“慢得多”。这对防止出现网络拥塞是一个非常好的方法。

为了防止拥塞窗口 $cwnd$ 增长过大引起网络拥塞，还需要设置一个慢开始门限 $ssthresh$ 状态变量（可以把门限 $ssthresh$ 的数值设置大些，例如达到发送窗口的最大容许值）。慢开

始门限 $ssthresh$ 的用法如下：

当 $cwnd < ssthresh$ 时，使用上述的慢开始算法。

当 $cwnd > ssthresh$ 时，停止使用慢开始算法而改用**拥塞避免**算法。

当 $cwnd = ssthresh$ 时，既可使用慢开始算法，也可使用拥塞避免算法。

拥塞避免算法的目的是让拥塞窗口 $cwnd$ 缓慢地增大（具体算法见[RFC 5681]）。执行算法后的结果大约是这样的：每经过一个往返时间 RTT ，发送方的拥塞窗口 $cwnd$ 的大小就加 1，而不是像慢开始阶段那样加倍增长。因此在拥塞避免阶段就称为“**加法增大**” AI (Additive Increase)，表明在拥塞避免阶段，拥塞窗口 $cwnd$ 按线性规律缓慢增长，比慢开始算法的拥塞窗口增长速率缓慢得多。

可以用曲线来说明 TCP 的拥塞窗口 $cwnd$ 是怎样随时间变化的（如图 5-25 所示）。但这里请特别注意横坐标采用的单位是往返时延 RTT 。在实际的互联网中，TCP 发送的每一个报文段的往返时延 RTT 都是不一样的（不会像图 5-24 中所画出的那样很理想的情况）。但在这里我们是讲解拥塞控制的原理，因此应当把图中的 RTT 理解为一个大致的时间，在这样的时间之内，发送方发出了一批报文段，并且都收到了接收方的确认。图 5-25 中的数字 ①至⑤是特别要注意的几个点。现假定 TCP 的发送窗口等于拥塞窗口。

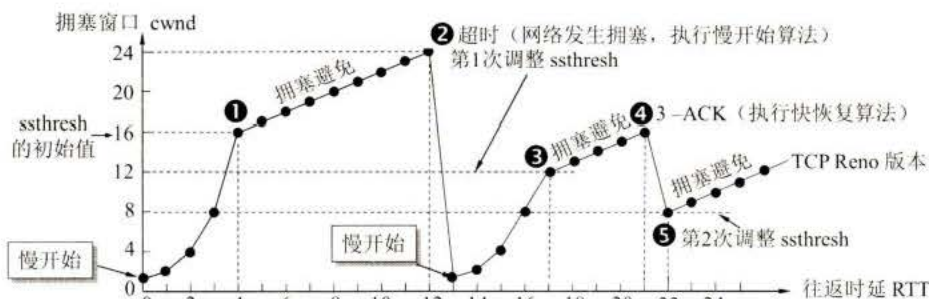


图 5-25 TCP 拥塞窗口 $cwnd$ 在拥塞控制时的变化情况

当 TCP 连接已建立后，把拥塞窗口 $cwnd$ 置为 1。在本例中，慢开始门限的初始值设置为 16 个报文段，即 $ssthresh = 16$ 。在执行慢开始算法阶段，每经过一个往返时间 RTT ，拥塞窗口 $cwnd$ 就加倍。当拥塞窗口 $cwnd$ 增长到慢开始门限值 $ssthresh$ 时（图中的点①，此时拥塞窗口 $cwnd = 16$ ），就改为执行拥塞避免算法，拥塞窗口按线性规律增长。但请注意，“拥塞避免”并非完全避免拥塞，而是让拥塞窗口增长得缓慢些，使网络不容易出现拥塞。

当拥塞窗口 $cwnd = 24$ 时，网络出现了超时（图中的点②），这就是网络发生拥塞的标志。于是调整门限值 $ssthresh = cwnd / 2 = 12$ ，同时设置拥塞窗口 $cwnd = 1$ ，执行慢开始算法。

按照慢开始算法，发送方每收到一个对新报文段的确认 ACK，就把拥塞窗口值加 1。当拥塞窗口 $cwnd = ssthresh = 12$ 时（图中的点③，这是 $ssthresh$ 第 1 次调整后的数值），改为执行拥塞避免算法，拥塞窗口按线性规律增大。

当拥塞窗口 $cwnd = 16$ 时（图中的点④），出现了一个新的情况，就是发送方一连收到 3 个对同一个报文段的重复确认（图中记为 3-ACK）。关于这个问题要解释如下。

有时，个别报文段会在网络中意外丢失，但实际上网络并未发生拥塞。如果发送方迟迟收不到确认，就会产生超时，并误认为网络发生了拥塞。这就导致发送方错误地启动慢开始，把拥塞窗口 $cwnd$ 又设置为 1，因而不必要地降低了传输效率。

采用快重传算法可以让发送方尽早知道发生了个别报文段的丢失。快重传算法首先要求接收方不要等待自己发送数据时才进行捎带确认，而是要立即发送确认，即使收到了失序的报文段也要立即发出对已收到的报文段的重复确认。如图 5-26 所示，接收方收到了 M_1 和 M_2 后都分别及时发出了确认。现假定接收方没有收到 M_3 但却收到了 M_4 。本来接收方可以什么都不做。但按照快重传算法，接收方必须立即发送对 M_2 的重复确认，以便让发送方及早知道接收方没有收到报文段 M_3 。发送方接着发送 M_5 和 M_6 。接收方收到后也仍要再次分别发出对 M_2 的重复确认。这样，发送方共收到了接收方的 4 个对 M_2 的确认，其中后 3 个都是重复确认。快重传算法规定，发送方只要一连收到 3 个重复确认，就可知道现在并未出现网络拥塞，而只是接收方少收到一个报文段 M_3 ，因而立即进行重传 M_3 （即“快重传”）。使用快重传可以使整个网络的吞吐量提高约 20%。

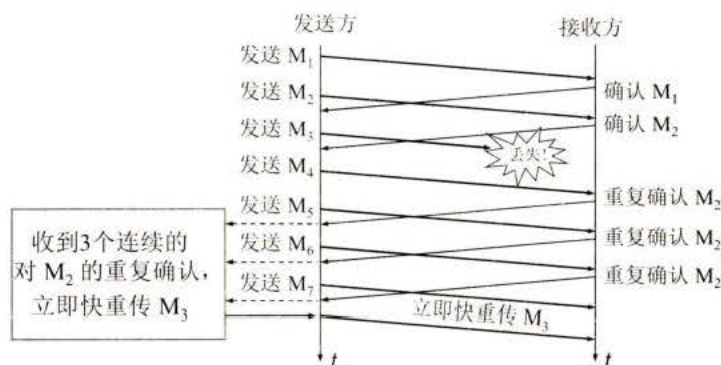


图 5-26 快重传的示意图

因此，在图 5-25 中的点④，发送方知道现在只是丢失了个别的报文段。于是不启动慢开始，而是执行快恢复算法。这时，发送方第 2 次调整门限值，使 $ssthresh = cwnd / 2 = 8$ ，同时设置拥塞窗口 $cwnd = ssthresh = 8$ （见图 5-25 中的点⑤），并开始执行拥塞避免算法。

在图 5-25 中还标注有“TCP Reno 版本”，表示区别于老的 TCP Tahoe 版本。

请注意，也有的快恢复实现是把快恢复开始时的拥塞窗口 $cwnd$ 值再增大一些（增大 3 个报文段的长度），即等于新的 $ssthresh + 3 \times MSS$ 。这样做的理由是：既然发送方收到 3 个重复的确认，就表明有 3 个分组已经离开了网络。这 3 个分组不再消耗网络的资源而是停留在接收方的缓存中（接收方发送出 3 个重复的确认就证明了这个事实）。可见现在网络中并不是堆积了分组而是减少了 3 个分组。因此可以适当把拥塞窗口扩大些。

从图 5-25 可以看出，在拥塞避免阶段，拥塞窗口是按照线性规律增大的，这就是前面提到过的加法增大 AI。而一旦出现超时或 3 个重复的确认，就要把门限值设置为当前拥塞窗口值的一半，并大大减小拥塞窗口的数值。这常称为“乘法减小”MD (Multiplicative Decrease)。二者合在一起就是所谓的 AIMD 算法。

采用这样的拥塞控制方法使得 TCP 的性能有明显的改进[STEV94][RFC 5681]。

根据以上所述，TCP 的拥塞控制可以归纳为图 5-27 的流程图。这个流程图就比图 5-25 所示的特例要更加全面些。例如，图 5-25 没有说明在慢开始阶段如果出现了超时（即出现了网络拥塞）或出现 3-ACK，发送方应采取什么措施。但从图 5-27 的流程图就可以很明确地知道发送方应采取的措施。

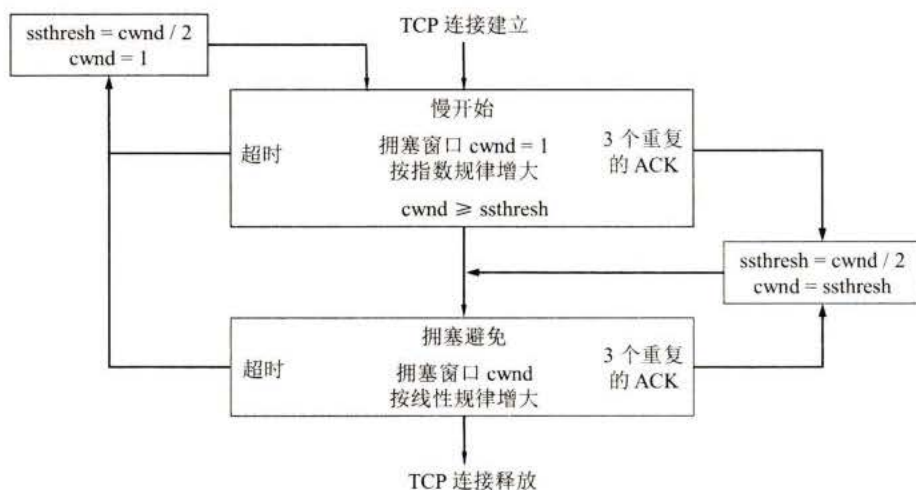


图 5-27 TCP 的拥塞控制的流程图

在这一节的开始我们就假定了接收方总是有足够大的缓存空间，因而发送窗口的大小由网络的拥塞程度来决定。但实际上接收方的缓存空间总是有限的。接收方根据自己的接收能力设定了接收方窗口 $rwnd$ ，并把这个窗口值写入 TCP 首部中的窗口字段，传送给发送方。因此，接收方窗口又称为通知窗口(advertised window)。因此，从接收方对发送方的流量控制的角度考虑，发送方的发送窗口一定不能超过对方给出的接收方窗口值 $rwnd$ 。

如果把本节所讨论的拥塞控制和接收方对发送方的流量控制一起考虑，那么很显然，发送方的窗口的上限值应当取为接收方窗口 $rwnd$ 和拥塞窗口 $cwnd$ 这两个变量中较小的一个，也就是说：

$$\text{发送方窗口的上限值} = \text{Min} [rwnd, cwnd] \quad (5-9)$$

式(5-9)指出：

当 $rwnd < cwnd$ 时，是接收方的接收能力限制发送方窗口的最大值。

反之，当 $cwnd < rwnd$ 时，则是网络的拥塞程度限制发送方窗口的最大值。

也就是说， $rwnd$ 和 $cwnd$ 中数值较小的一个，控制了发送方发送数据的速率。

5.8.3 主动队列管理 AQM

上一节讨论的 TCP 拥塞控制并没有和网络层采取的策略联系起来。其实，它们之间有着密切的关系。

例如，假定一个路由器对某些分组的处理时间特别长，那么这就可能使这些分组中的数据部分（即 TCP 报文段）经过很长时间才能到达终点，结果引起发送方对这些报文段的重传。根据前面所讲的，重传会使 TCP 连接的发送端认为在网络中发生了拥塞。于是在 TCP 的发送端就采取了拥塞控制措施，但实际上网络并没有发生拥塞。

网络层的策略对 TCP 拥塞控制影响最大的就是路由器的分组丢弃策略。在最简单的情况下，路由器的队列通常都按照“先进先出”FIFO (First In First Out) 的规则处理到来的分组。由于队列长度总是有限的，因此当队列已满时，以后再到达的所有分组（如果能够继

续排队，这些分组都将排在队列的尾部）将都被丢弃。这就叫作**尾部丢弃策略**(tail-drop policy)。

路由器的尾部丢弃往往会导致一连串分组的丢失，这就使发送方出现超时重传，使 TCP 进入拥塞控制的慢开始状态，结果使 TCP 连接的发送方突然把数据的发送速率降低到很小的数值。更为严重的是，在网络中通常有很多的 TCP 连接（它们有不同的源点和终点），这些连接中的报文段通常是复用在网络层的 IP 数据报中传送的。在这种情况下，若发生了路由器中的尾部丢弃，就可能会同时影响到很多条 TCP 连接，结果使这许多 TCP 连接在**同一时间**突然都进入到慢开始状态。这在 TCP 的术语中称为**全局同步**(global synchronization)。全局同步使得全网的通信量突然下降了很多，而在网络恢复正常后，其通信量又突然增大很多。

为了避免发生网络中的全局同步现象，在 1998 年提出了**主动队列管理** AQM (Active Queue Management)。所谓“主动”就是不要等到路由器的队列长度已经达到最大值时才不得不丢弃后面到达的分组。这样就太被动了。应当在队列长度达到某个值得警惕的数值时（即当网络拥塞有了某些拥塞征兆时），就主动丢弃到达的分组。这样就提醒了发送方放慢发送的速率，因而有可能使网络拥塞的程度减轻，甚至不出现网络拥塞。AQM 可以有不同实现方法，其中曾流行多年的就是**随机早期检测** RED (Random Early Detection)。RED 还有几个不同的名称，如 Random Early Drop 或 Random Early Discard（随机早期丢弃）。

实现 RED 时需要使路由器维持两个参数，即队列长度最小门限和最大门限。当每一个分组到达时，RED 就按照规定的算法先计算当前的平均队列长度。

(1) 若平均队列长度小于最小门限，则把新到达的分组放入队列进行排队。

(2) 若平均队列长度超过最大门限，则把新到达的分组丢弃。

(3) 若平均队列长度在最小门限和最大门限之间，则按照某一丢弃概率 p 把新到达的分组丢弃（这就体现了丢弃分组的随机性）。

由此可见，RED 不是等到已经发生网络拥塞后才把所有在队列尾部的分组全部丢弃，而是在检测到网络拥塞的**早期征兆**时（即路由器的平均队列长度达到一定数值时），就以概率 p 丢弃个别的分组，让拥塞控制只在个别的 TCP 连接上进行，因而避免发生全局性的拥塞控制。

在 RED 的操作中，最难处理的就是丢弃概率 p 的选择，因为 p 并不是个常数。对每一个到达的分组，都必须计算丢弃概率 p 的数值。IETF 曾经推荐在互联网中的路由器使用 RED 机制[RFC 2309]，但多年的实践证明，RED 的使用效果并不太理想。因此，在 2015 年公布的 RFC 7567 已经把过去的 RFC 2309 列为“陈旧的”，并且不再推荐使用 RED。对路由器进行主动队列管理 AQM 仍是必要的。AQM 实际上就是对路由器中的分组排队进行智能管理，而不是简单地把队列的尾部丢弃。现在已经有几种不同的算法来代替旧的 RED，但都还在实验阶段。目前还没有一种算法能够成为 IETF 的标准，读者可注意这方面的进展。

5.9 TCP 的运输连接管理

TCP 是面向连接的协议。运输连接是用来传送 TCP 报文的。TCP 运输连接的建立和释

放是每一次面向连接的通信中必不可少的过程。因此，运输连接就有三个阶段，即：连接建立、数据传送和连接释放。运输连接的管理就是使运输连接的建立和释放都能正常地进行。

在 TCP 连接建立过程中要解决以下三个问题：

- (1) 要使每一方能够确知对方的存在。
- (2) 要允许双方协商一些参数（如最大窗口值、是否使用窗口扩大选项和时间戳选项以及服务质量等）。
- (3) 能够对运输实体资源（如缓存大小、连接表中的项目等）进行分配。

TCP 连接的建立采用客户服务器方式。主动发起连接建立的应用进程叫作**客户(client)**，而被动等待连接建立的应用进程叫作**服务器(server)**。

5.9.1 TCP 的连接建立

TCP 建立连接的过程叫作握手，握手需要在客户和服务器之间交换三个 TCP 报文段。图 5-28 画出了三报文握手^①建立 TCP 连接的过程。

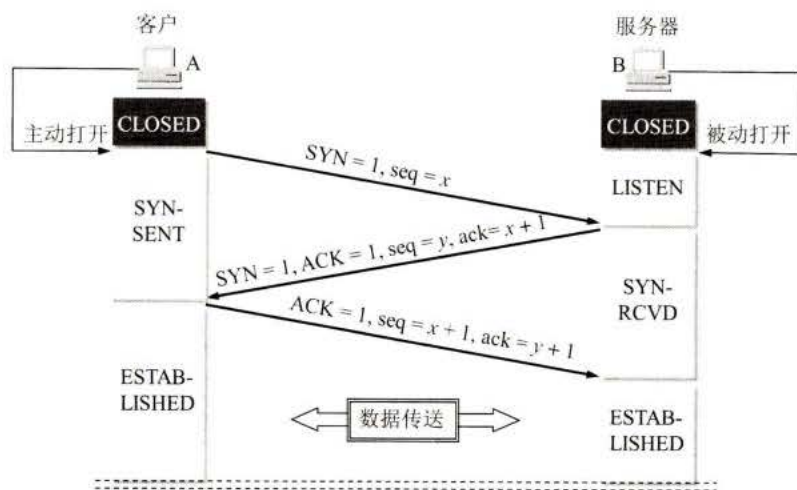


图 5-28 用三报文握手建立 TCP 连接

假定主机 A 运行的是 TCP 客户程序，而 B 运行 TCP 服务器程序。最初两端的 TCP 进程都处于 CLOSED（关闭）状态。图中在主机下面的方框分别是 TCP 进程所处的状态。请注意，在本例中，A 主动打开连接，而 B 被动打开连接。

一开始，B 的 TCP 服务器进程先创建**传输控制块 TCB^②**，准备接受客户进程的连接请求。然后服务器进程就处于 LISTEN（收听）状态，等待客户的连接请求。如有，即做出响应。

① 注：三报文握手是本教材首次采用的译名。在 RFC 793（TCP 标准的文档）中使用的名称是 three way handshake，但这个名称很难译为准确的中文。例如，以前本教材曾采用“三次握手”这个广为流行的译名。其实这是在一次握手过程中交换了三个报文，而并不是进行了三次握手（这有点像两个人见面进行一次握手时，他们的手上下摇晃了三次，但这并非进行了三次握手）。最近再次重新阅读了 RFC 793 文档，发现有这样的表述：“three way (three message) handshake”。可见采用“三报文握手”这样的译名，在意思的表达上应当是比较准确的。请注意，handshake 使用的是单数而不是复数，表明只是一次握手。

② 注：传输控制块 TCB (Transmission Control Block) 存储了每一个连接中的一些重要信息，如：TCP 连接表、指向发送和接收缓存的指针、指向重传队列的指针、当前的发送和接收序号，等等。

扫一扫



视频讲解

A 的 TCP 客户进程也是首先创建**传输控制块 TCB**。然后，在打算建立 TCP 连接时，向 B 发出连接请求报文段，这时首部中的同步位 $\text{SYN} = 1$ ，同时选择一个初始序号 $\text{seq} = x$ 。TCP 规定，SYN 报文段（即 $\text{SYN} = 1$ 的报文段）不能携带数据，但要**消耗掉一个序号**。这时，TCP 客户进程进入 SYN-SENT（同步已发送）状态。

B 收到连接请求报文段后，如同意建立连接，则向 A 发送确认。在确认报文段中应把 SYN 位和 ACK 位都置 1，确认号是 $\text{ack} = x + 1$ ，同时也为自己选择一个初始序号 $\text{seq} = y$ 。请注意，这个报文段也不能携带数据，但同样**要消耗掉一个序号**。这时 TCP 服务器进程进入 SYN-RCVD（同步收到）状态。

TCP 客户进程收到 B 的确认后，还要向 B 给出确认。确认报文段的 ACK 置 1，确认号 $\text{ack} = y + 1$ ，而自己的序号 $\text{seq} = x + 1$ 。TCP 的标准规定，ACK 报文段可以携带数据。但**如果不携带数据则不消耗序号**，在这种情况下，下一个数据报文段的序号仍是 $\text{seq} = x + 1$ 。这时，TCP 连接已经建立，A 进入 ESTABLISHED（已建立连接）状态。

当 B 收到 A 的确认后，也进入 ESTABLISHED 状态。

上面给出的连接建立过程叫作**三报文握手**。请注意，在图 5-28 中 B 发送给 A 的报文段，也可拆成两个报文段。可以先发送一个确认报文段（ $\text{ACK} = 1, \text{ack} = x + 1$ ），然后再发送一个同步报文段（ $\text{SYN} = 1, \text{seq} = y$ ）。这样的过程就变成了**四报文握手**，但效果是一样的。

为什么 A 最后还要发送一次确认呢？这主要是为了防止已失效的连接请求报文段突然又传送到了 B，因而产生错误。

所谓“已失效的连接请求报文段”是这样产生的。考虑一种正常情况，A 发出连接请求，但因连接请求报文丢失而未收到确认。于是 A 再重传一次连接请求。后来收到了确认，建立了连接。数据传输完毕后，就释放了连接。A 共发送了两个连接请求报文段，其中第一个丢失，第二个到达了 B，没有“已失效的连接请求报文段”。

现假定出现一种异常情况，即 A 发出的第一个连接请求报文段并没有丢失，而是在某些网络节点长时间滞留了，以致延误到连接释放以后的某个时间才到达 B。本来这是一个早已失效的报文段。但 B 收到此失效的连接请求报文段后，就误认为是 A 又发出一次新的连接请求。于是就向 A 发出确认报文段，同意建立连接。假定不采用报文握手，那么只要 B 发出确认，新的连接就建立了。

由于现在 A 并没有发出建立连接的请求，因此不会理睬 B 的确认，也不会向 B 发送数据。但 B 却以为新的运输连接已经建立了，并一直等待 A 发来数据。B 的许多资源就这样白白浪费了。

采用三报文握手的办法，可以防止上述现象的发生。例如在刚才的异常情况下，A 不会向 B 的确认发出确认。B 由于收不到确认，就知道 A 并没有要求建立连接。

5.9.2 TCP 的连接释放

TCP 连接释放过程比较复杂，我们仍结合双方状态的变化来阐明连接释放的过程。

数据传输结束后，通信的双方都可释放连接。现在 A 和 B 都处于 ESTABLISHED 状态（如图 5-29 所示）。A 的应用进程先向其 TCP 发出连接释放报文段，并停止再发送数据，主动关闭 TCP 连接。A 把连接释放报文段首部的终止

扫一扫



视频讲解

控制位 FIN 置 1，其序号 $\text{seq} = u$ ，它等于前面已传送过的数据的最后一个字节的序号加 1。这时 A 进入 FIN-WAIT-1（终止等待 1）状态，等待 B 的确认。请注意，TCP 规定，FIN 报文段即使不携带数据，它也消耗掉一个序号。

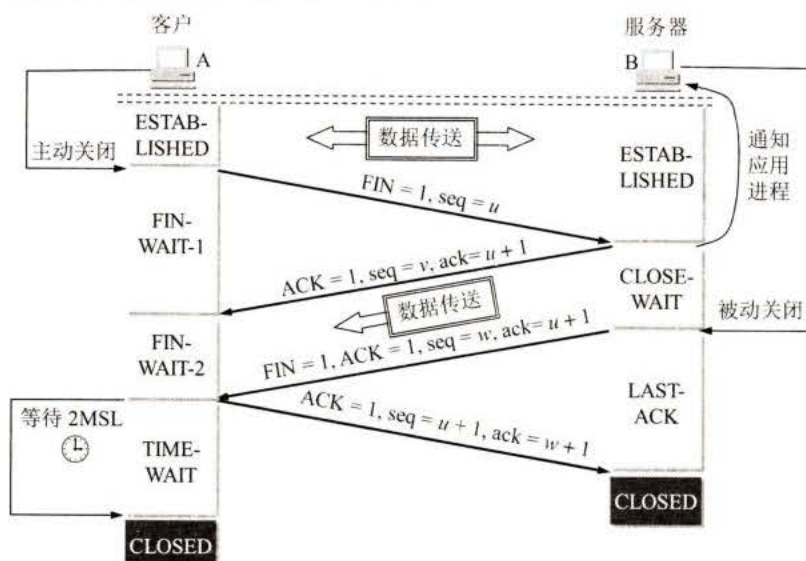


图 5-29 TCP 连接释放的过程

B 收到连接释放报文段后即发出确认，确认号是 $\text{ack} = u + 1$ ，而这个报文段自己的序号是 v ，等于 B 前面已传送过的数据的最后一个字节的序号加 1。然后 B 就进入 CLOSE-WAIT（关闭等待）状态。TCP 服务器进程这时应通知高层应用进程，因而从 A 到 B 这个方向的连接就释放了，这时的 TCP 连接处于半关闭(half-close)状态，即 A 已经没有数据要发送了，但 B 若发送数据，A 仍要接收。也就是说，从 B 到 A 这个方向的连接并未关闭，这个状态可能会持续一段时间。

A 收到来自 B 的确认后，就进入 FIN-WAIT-2（终止等待 2）状态，等待 B 发出的连接释放报文段。

若 B 已经没有要向 A 发送的数据，其应用进程就通知 TCP 释放连接。这时 B 发出的连接释放报文段必须使 $\text{FIN} = 1$ 。现假定 B 的序号为 w （在半关闭状态 B 可能又发送了一些数据）。B 还必须重复上次已发送过的确认号 $\text{ack} = u + 1$ 。这时 B 就进入 LAST-ACK（最后确认）状态，等待 A 的确认。

A 在收到 B 的连接释放报文段后，必须对此发出确认。在确认报文段中把 ACK 置 1，确认号 $\text{ack} = w + 1$ ，而自己的序号是 $\text{seq} = u + 1$ （根据 TCP 标准，前面发送过的 FIN 报文段要消耗一个序号）。然后进入到 TIME-WAIT（时间等待）状态。请注意，现在 TCP 连接还没有释放掉。必须经过时间等待计时器(TIME-WAIT timer)设置的时间 2MSL 后，A 才进入到 CLOSED 状态。时间 MSL 叫作最长报文段寿命(Maximum Segment Lifetime)，RFC 793 建议设为 2 分钟。但这完全是从工程上来考虑的，对于现在的网络， $\text{MSL} = 2$ 分钟可能太长了一些。因此 TCP 允许不同的实现可根据具体情况使用更小的 MSL 值。因此，从 A 进入到 TIME-WAIT 状态后，要经过 4 分钟才能进入到 CLOSED 状态，才能开始建立下一个新

的连接。当 A 撤销相应的传输控制块 TCB 后，就结束了这次的 TCP 连接。

为什么 A 在 TIME-WAIT 状态必须等待 2MSL 的时间呢？这有两个理由。

第一，为了保证 A 发送的最后一个 ACK 报文段能够到达 B。这个 ACK 报文段有可能丢失，因而使处在 LAST-ACK 状态的 B 收不到对已发送的 FIN + ACK 报文段的确认。B 会超时重传这个 FIN + ACK 报文段，而 A 就能在 2MSL 时间内收到这个重传的 FIN + ACK 报文段。接着 A 重传一次确认，重新启动 2MSL 计时器。最后，A 和 B 都正常进入到 CLOSED 状态。如果 A 在 TIME-WAIT 状态不等待一段时间，而是在发送完 ACK 报文段后立即释放连接，那么就无法收到 B 重传的 FIN + ACK 报文段，因而也不会再发送一次确认报文段。这样，B 就无法按照正常步骤进入 CLOSED 状态。

第二，防止上一节提到的“已失效的连接请求报文段”出现在本连接中。A 在发送完最后一个 ACK 报文段后，再经过时间 2MSL，就可以使本连接持续的时间内所产生的所有报文段都从网络中消失。这样就可以使下一个新的连接中不会出现这种旧的连接请求报文段。

B 只要收到了 A 发出的确认，就进入 CLOSED 状态。同样，B 在撤销相应的传输控制块 TCB 后，就结束了这次的 TCP 连接。我们注意到，B 结束 TCP 连接的时间要比 A 早一些。

上述的 TCP 连接释放过程是四报文握手。

除时间等待计时器外，TCP 还设有一个保活计时器(keepalive timer)。设想有这样的情况：客户已主动与服务器建立了 TCP 连接。但后来客户端的主机突然出现故障。显然，服务器以后就不能再收到客户发来的数据。因此，应当有措施使服务器不要再白白等待下去。这就是使用保活计时器。服务器每收到一次客户的数据，就重新设置保活计时器，时间的设置通常是两小时。若两小时没有收到客户的数据，服务器就发送一个探测报文段，以后则每隔 75 秒钟发送一次。若一连发送 10 个探测报文段后仍无客户的响应，服务器就认为客户端出了故障，接着就关闭这个连接。

5.9.3 TCP 的有限状态机

为了更清晰地看出 TCP 连接的各种状态之间的关系，图 5-30 给出了 TCP 的有限状态机。图中每一个方框是 TCP 可能具有的状态。每个方框中的大写英文字符串是 TCP 标准所使用的 TCP 连接状态名。状态之间的箭头表示可能发生的状态变迁。箭头旁边的字，表明引起这种变迁的原因，或表明发生状态变迁后又出现什么动作。请注意图中有三种不同的箭头。粗实线箭头表示对客户进程的正常变迁。粗虚线箭头表示对服务器进程的正常变迁。另一种细线箭头表示异常变迁。

我们可以把图 5-30 和前面的图 5-28、图 5-29 对照起来看。在图 5-28 和图 5-29 中左边客户进程从上到下的状态变迁，就是图 5-30 中粗实线箭头所指的状态变迁。而在图 5-28 和 5-29 右边服务器进程从上到下的状态变迁，就是图 5-30 中粗虚线箭头所指的状态变迁。

还有一些状态变迁，例如连接建立过程中的从 LISTEN 到 SYN-SENT 和从 SYN-SENT 到 SYN-RCVD。读者可分析在什么情况下会出现这样的变迁（见习题 5-43）。

扫一扫



视频讲解

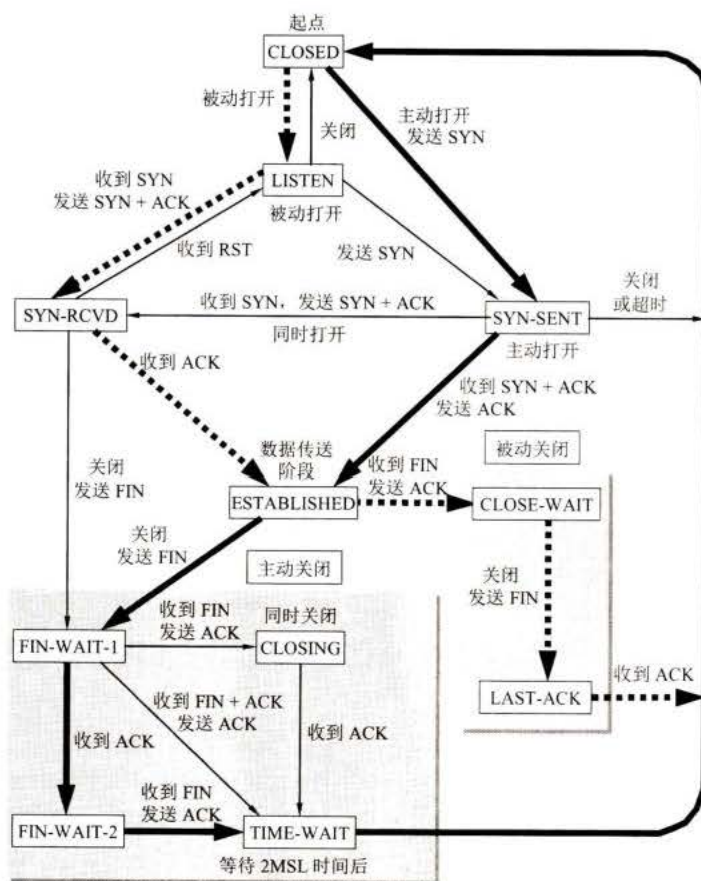


图 5-30 TCP 的有限状态机

本章的重要概念

- 运输层提供应用进程间的逻辑通信，也就是说，运输层之间的通信并不是真正在两个运输层之间直接传送数据。运输层向应用层屏蔽了下面网络的细节（如网络拓扑、所采用的路由选择协议等），它使应用进程看见的就是好像在两个运输层实体之间有一条端到端的逻辑通信信道。
- 网络层为主机之间提供逻辑通信，而运输层为应用进程之间提供端到端的逻辑通信。
- 运输层有两个主要的协议：TCP 和 UDP。它们都有复用和分用，以及检错的功能。当运输层采用面向连接的 TCP 协议时，尽管下面的网络是不可靠的（只提供尽最大努力服务），但这种逻辑通信信道就相当于一全双工通信的可靠信道。当运输层采用无连接的 UDP 协议时，这种逻辑通信信道仍然是一条不可靠信道。
- 运输层用一个 16 位端口号来标志一个端口。端口号只具有本地意义，它只是为了标志本计算机应用层中的各个进程在和运输层交互时的层间接口。在互联网的不同计算机中，相同的端口号是没有关联的。
- 两台计算机中的进程要互相通信，不仅要知道对方的 IP 地址（为了找到对方的计算机），而且还要知道对方的端口号（为了找到对方计算机中的应用进程）。

- 运输层的端口号分为服务器端使用的端口号（0 ~ 1023 指派给熟知端口，1024 ~ 49151 是登记端口号）和客户端暂时使用的端口号（49152 ~ 65535）。
- UDP 的主要特点是：(1) 无连接；(2) 尽最大努力交付；(3) 面向报文；(4) 无拥塞控制；(5) 支持一对一、一对多、多对一和多对多的交互通信；(6) 首部开销小（只有四个字段：源端口、目的端口、长度和检验和）。
- TCP 的主要特点是：(1) 面向连接；(2) 每一条 TCP 连接只能是点对点的（一对一）；(3) 提供可靠交付的服务；(4) 提供全双工通信；(5) 面向字节流。
- TCP 用主机的 IP 地址加上主机上的端口号作为 TCP 连接的端点。这样的端点就叫做套接字(socket)或插口。套接字用（IP 地址：端口号）来表示。
- 停止等待协议能够在不可靠的传输网络上实现可靠的通信。每发送完一个分组就停止发送，等待对方的确认。在收到确认后再发送下一个分组。分组需要进行编号。
- 超时重传是指只要超过了一段时间仍然没有收到确认，就重传前面发送过的分组（认为刚才发送的分组丢失了）。因此每发送完一个分组需要设置一个超时计时器，其重传时间应比数据在分组传输的平均往返时间更长一些。这种自动重传方式常称为自动重传请求 ARQ。
- 在停止等待协议中，若接收方收到重复分组，就丢弃该分组，但同时还要发送确认。
- 连续 ARQ 协议可提高信道利用率。发送方维持一个发送窗口，凡位于发送窗口内的分组都可连续发送出去，而不需要等待对方的确认。接收方一般采用累积确认，对按序到达的最后一个分组发送确认，表明到这个分组为止的所有分组都已正确收到了。
- TCP 报文段首部的 20 个字节是固定的，后面有 4N 字节是根据需要而增加的选项（N 是整数）。在一个 TCP 连接中传送的字节流中的每一个字节都按顺序编号。首部中的序号字段值则指的是本报文段所发送的数据的第一个字节的序号。
- TCP 首部中的确认号是期望收到对方下一个报文段的第一个数据字节的序号。若确认号为 N，则表明：到序号 N-1 为止的所有数据都已正确收到。
- TCP 首部中的窗口字段指出了现在允许对方发送的数据量。窗口值是经常动态变化着的。
- TCP 使用滑动窗口机制。发送窗口里面的序号表示允许发送的序号。发送窗口后沿的后面部分表示已发送且已收到了确认，而发送窗口前沿的前面部分表示不允许发送。发送窗口后沿的变化情况有两种可能，即不动（没有收到新的确认）和前移（收到了新的确认）。发送窗口前沿通常是不断向前移动的。
- 流量控制就是让发送方的发送速率不要太快，要让接收方来得及接收。
- 在某段时间，若对网络中某一资源的需求超过了该资源所能提供的可用部分，网络的性能就要变坏。这种情况就叫作拥塞。拥塞控制就是防止过多的数据注入到网络中，这样可以使网络中的路由器或链路不至于过载。
- 流量控制是一个端到端的问题，是接收端抑制发送端发送数据的速率，以便使接收端来得及接收。拥塞控制是一个全局性的过程，涉及所有的主机、所有的路由器，以及与降低网络传输性能有关的所有因素。
- 为了进行拥塞控制，TCP 的发送方要维持一个拥塞窗口 cwnd 的状态变量。拥塞窗

口的大小取决于网络的拥塞程度，并且动态地在变化。发送方让自己的发送窗口取为拥塞窗口和接收方的接收窗口中较小的一个。

- TCP 的拥塞控制采用了四种算法，即慢开始、拥塞避免、快重传和快恢复。在网络层，也可以使路由器采用适当的分组丢弃策略（如主动队列管理 AQM），以减少网络拥塞的发生。
- 运输连接有三个阶段，即：连接建立、数据传送和连接释放。
- 主动发起 TCP 连接建立的应用进程叫作客户，而被动等待连接建立的应用进程叫作服务器。TCP 的连接建立采用三报文握手机制。服务器要确认客户的连接请求，然后客户要对服务器的确认进行确认。
- TCP 的连接释放采用四报文握手机制。任何一方都可以在数据传送结束后发出连接释放的通知，待对方确认后就进入半关闭状态。当另一方也没有数据再发送时，则发送连接释放通知，对方确认后就完全关闭了 TCP 连接。

习题

- 5-01** 试说明运输层在协议栈中的地位和作用。运输层的通信和网络层的通信有什么重要的区别？为什么运输层是必不可少的？
- 5-02** 网络层提供数据报或虚电路服务对上面的运输层有何影响？
- 5-03** 当应用程序使用面向连接的 TCP 和无连接的 IP 时，这种传输是面向连接的还是无连接的？
- 5-04** 试画图解释运输层的复用。画图说明许多个运输用户复用到一条运输连接上，而这条运输连接又复用到 IP 数据报上。
- 5-05** 试举例说明有些应用程序愿意采用不可靠的 UDP，而不愿意采用可靠的 TCP。
- 5-06** 接收方收到有差错的 UDP 用户数据报时应如何处理？
- 5-07** 如果应用程序愿意使用 UDP 完成可靠传输，这可能吗？请说明理由。
- 5-08** 为什么说 UDP 是面向报文的，而 TCP 是面向字节流的？
- 5-09** 端口的作用是什么？为什么端口号要划分为三种？
- 5-10** 试说明运输层中伪首部的作用。
- 5-11** 某个应用进程使用运输层的用户数据报 UDP，然后继续向下交给 IP 层后，又封装成 IP 数据报。既然都是数据报，是否可以跳过 UDP 而直接交给 IP 层？哪些功能 UDP 提供了但 IP 没有提供？
- 5-12** 一个应用程序用 UDP，到了 IP 层把数据报再划分为 4 个数据报片发送出去。结果前两个数据报片丢失，后两个到达目的站。过了一段时间应用程序重传 UDP，而 IP 层仍然划分为 4 个数据报片来传送。结果这次前两个到达目的站而后两个丢失。试问：在目的站能否将这两次传输的 4 个数据报片组装成为完整的数据报？假定目的站第一次收到的后两个数据报片仍然保存在目的站的缓存中。
- 5-13** 一个 UDP 用户数据报的数据字段为 8192 字节。在链路层要使用以太网来传送。试问应当划分为几个 IP 数据报片？说明每一个 IP 数据报片的数据字段长度和片偏移字段的值。
- 5-14** 一个 UDP 用户数据报的首部的十六进制表示是：06 32 00 45 00 1C E2 17。试求源端口、目的端口、用户数据报的总长度、数据部分长度。这个用户数据报是从客户发

送给服务器还是从服务器发送给客户？使用 UDP 的这个服务器程序是什么？

- 5-15** 使用 TCP 对实时话音数据的传输会有什么问题？使用 UDP 在传送数据文件时会有什么问题？
- 5-16** 在停止等待协议中如果不使用编号是否可行？为什么？
- 5-17** 在停止等待协议中，如果收到重复的报文段时不予理睬（即悄悄地丢弃它而其他什么也不做）是否可行？试举出具体例子说明理由。
- 5-18** 假定在运输层使用停止等待协议。发送方发送报文段 M_0 后在设定的时间内未收到确认，于是重传 M_0 ，但 M_0 又迟迟不能到达接收方。不久，发送方收到了迟到的对 M_0 的确认，于是发送下一个报文段 M_1 ，不久就收到了对 M_1 的确认。接着发送方发送新的报文段 M_0 ，但这个新的 M_0 在传送过程中丢失了。正巧，一开始就滞留在网络中的 M_0 现在到达接收方。接收方无法分辨 M_0 是旧的。于是收下 M_0 ，并发送确认。显然，接收方后来收到的 M_0 是重复的，协议失败了。
试画出类似于图 5-9 所示的双方交换报文段的过程。
- 5-19** 试证明：当用 n 比特进行分组的编号时，若接收窗口等于 1（即只能按序接收分组），则仅在发送窗口不超过 $2^n - 1$ 时，连续 ARQ 协议才能正确运行。窗口单位是分组。
- 5-20** 在连续 ARQ 协议中，若发送窗口等于 7，则发送端在开始时可连续发送 7 个分组。因此，在每一分组发出后，都要置一个超时计时器。现在计算机里只有一个硬时钟。设这 7 个分组发出的时间分别为 $t_0, t_1, \dots, t_6$ ，且 t_{out} 都一样大。试问如何实现这 7 个超时计时器（这叫软时钟法）？
- 5-21** 假定使用连续 ARQ 协议，发送窗口大小是 3，而序号范围是 $[0, 15]$ ，而传输媒体保证在接收方能够按序收到分组。在某一时刻，在接收方，下一个期望收到的序号是 5。试问：
(1) 在发送方的发送窗口中可能出现的序号组合有哪些？
(2) 接收方已经发送出的、但在网络中（即还未到达发送方）的确认分组可能有哪些？说明这些确认分组是用来确认哪些序号的分组。
- 5-22** 主机 A 向主机 B 发送一个很长的文件，其长度为 L 字节。假定 TCP 使用的 MSS 为 1460 字节。
(1) 在 TCP 的序号不重复使用的条件下， L 的最大值是多少？
(2) 假定使用上面计算出的文件长度，而运输层、网络层和数据链路层所用的首部开销共 66 字节，链路的数据率为 10 Mbit/s，试求这个文件所需的最短发送时间。
- 5-23** 主机 A 向主机 B 连续发送了两个 TCP 报文段，其序号分别是 70 和 100。试问：
(1) 第一个报文段携带了多少字节的数据？
(2) 主机 B 收到第一个报文段后发回的确认中的确认号应当是多少？
(3) 如果 B 收到第二个报文段后发回的确认中的确认号是 180，试问 A 发送的第二个报文段中的数据有多少字节？
(4) 如果 A 发送的第一个报文段丢失了，但第二个报文段到达了 B。B 在第二个报文段到达后向 A 发送确认。试问这个确认号应为多少？
- 5-24** 一个 TCP 连接下面使用 256 kbit/s 的链路，其端到端时延为 128 ms。经测试，发现吞吐量只有 120 kbit/s。试问发送窗口 W 是多少？（提示：可以有两种答案，取决于接收端发出确认的时机。）

- 5-25 为什么在 TCP 首部中要把 TCP 的端口号放入最开始的 4 个字节?
- 5-26 为什么在 TCP 首部中有一个首部长度字段, 而 UDP 的首部中就没有这个字段?
- 5-27 一个 TCP 报文段的数据部分最多为多少个字节? 为什么? 如果用户要传送的数据的字节长度超过 TCP 报文段中的序号字段可能编出的最大序号, 问还能否用 TCP 来传送?
- 5-28 主机 A 向主机 B 发送 TCP 报文段, 首部中的源端口是 m 而目的端口是 n 。当 B 向 A 发送回信时, 其 TCP 报文段的首部中的源端口和目的端口分别是什么?
- 5-29 在使用 TCP 传送数据时, 如果有一个确认报文段丢失了, 也不一定会引起与该确认报文段对应的数据的重传。试说明理由。
- 5-30 设 TCP 使用的最大窗口为 65535 字节, 而传输信道不产生差错, 带宽也不受限制。若报文段的平均往返时间为 20 ms, 问所能得到的最大吞吐量是多少?
- 5-31 通信信道带宽为 1 Gbit/s, 端到端传播时延为 10 ms。TCP 的发送窗口为 65535 字节。试问: 可能达到的最大吞吐量是多少? 信道的利用率是多少?
- 5-32 什么是 Karn 算法? 在 TCP 的重传机制中, 若不采用 Karn 算法, 而是在收到确认时都认为是重传报文段的确认, 那么由此得出的往返时间样本和重传时间都会偏小。试问: 重传时间最后会减小到什么程度?
- 5-33 假定 TCP 在开始建立连接时, 发送方设定超时重传时间 $RTO = 6$ 秒。
- (1) 当发送方收到对方的连接确认报文段时, 测量出 RTT 样本值为 1.5 秒。试计算现在的 RTO 值。
 - (2) 当发送方发送数据报文段并收到确认时, 测量出 RTT 样本值为 2.5 秒。试计算现在的 RTO 值。
- 5-34 已知第一次测得 TCP 的往返时间 RTT 是 30 ms。接着收到了三个确认报文段, 用它们测量出的往返时间样本 RTT 分别是: 26 ms, 32 ms 和 24 ms。设 $\alpha = 0.1$ 。试计算每一次新的加权平均往返时间值 RTT_s 。讨论所得出的结果。
- 5-35 用 TCP 通过速率为 1 Gbit/s 的链路传送一个 10 MB 的文件。假定链路的往返时延 $RTT = 50$ ms。TCP 选用了窗口扩大选项, 使窗口达到可选用的最大值。在接收端, TCP 的接收窗口为 1 MB (保持不变), 而发送端采用拥塞控制算法, 从慢开始传送。假定拥塞窗口以分组为单位计算, 在一开始发送 1 个分组, 而每个分组长度都是 1 KB。假定网络不会发生拥塞和分组丢失, 并且发送端发送数据的速率足够快, 因此发送时延可以忽略不计, 而接收端每一次收完一批分组后就立即发送确认 ACK 分组。
- (1) 经过多少个 RTT 后, 发送窗口大小达到 1 MB?
 - (2) 发送端把整个 10 MB 文件传送成功共需要经过多少个 RTT? 传送成功是指发送完整个文件, 并收到所有的确认。TCP 扩大的窗口够用吗?
 - (3) 根据整个文件发送成功所花费的时间 (包括收到所有的确认), 计算此传输链路的有效吞吐量。链路带宽的利用率是多少?
- 5-36 假定 TCP 采用一种仅使用线性增大和乘法减小的简单拥塞控制算法, 而不使用慢开始。发送窗口不采用字节为计算单位, 而是使用分组 pkt 为计算单位。在一开始时发送窗口为 1 pkt。假定分组的发送时延非常小, 可以忽略不计。所有产生的时延就是传播时延。假定发送窗口总是小于接收窗口。接收端每收到一组分组后, 就立即发回确认 ACK。假定分组的编号为 i , 在一开始发送的是 $i = 1$ 的分组。以后当 $i = 9, 25,$

30, 38 和 50 时, 发生了分组的丢失。再假定分组的超时重传时间正好是下一个 RTT 开始的时间。试画出拥塞窗口 (也就是发送窗口) 与 RTT 的关系曲线, 画到发送第 51 个分组为止。

5-37 在 TCP 的拥塞控制中, 什么是慢开始、拥塞避免、快重传和快恢复算法? 这里每一种算法各起什么作用? “乘法减小”和“加法增大”各用在什么情况下?

5-38 设 TCP 的 $ssthresh$ 的初始值为 8 (单位为报文段)。当拥塞窗口上升到 12 时网络发生了超时, TCP 使用慢开始和拥塞避免。试分别求出 $RTT = 1$ 到 $RTT = 15$ 的各拥塞窗口大小。你能说明拥塞窗口每一次变化的原因吗?

5-39 TCP 的拥塞窗口 $cwnd$ 大小与 RTT 的关系如下所示:

cwnd	1	2	4	8	16	32	33	34	35	36	37	38	39
RTT	1	2	3	4	5	6	7	8	9	10	11	12	13
cwnd	40	41	42	21	22	23	24	25	26	1	2	4	8
RTT	14	15	16	17	18	19	20	21	22	23	24	25	26

(1) 试画出如图 5-25 所示的拥塞窗口与 RTT 的关系曲线。

(2) 指明 TCP 工作在慢开始阶段的时间间隔。

(3) 指明 TCP 工作在拥塞避免阶段的时间间隔。

(4) 在 $RTT = 16$ 和 $RTT = 22$ 之后发送方是通过收到三个重复的确认还是通过超时检测到丢失了报文段?

(5) 在 $RTT = 1$ 、 $RTT = 18$ 和 $RTT = 24$ 时, 门限 $ssthresh$ 分别被设置为多大?

(6) 在 RTT 等于多少时发送出第 70 个报文段?

(7) 假定在 $RTT = 26$ 之后收到了三个重复的确认, 因而检测出了报文段的丢失, 那么拥塞窗口 $cwnd$ 和门限 $ssthresh$ 应设置为多大?

5-40 TCP 在进行流量控制时, 以分组的丢失作为产生拥塞的标志。有没有不是因拥塞而引起分组丢失的情况? 如有, 请举出三种情况。

5-41 用 TCP 传送 512 字节的数据。设窗口为 100 字节, 而 TCP 报文段每次也是传送 100 字节的数据。再设发送方和接收方的起始序号分别选为 100 和 200, 试画出类似于图 5-28 的工作示意图。从连接建立阶段到连接释放都要画上 (可不考虑传播时延)。

5-42 在图 5-29 中所示的连接释放过程中, 在 ESTABLISHED 状态下, 服务器进程能否先不发送 $ack = u + 1$ 的确认? (因为后面要发送的连接释放报文段中仍有 $ack = u + 1$ 这一信息。)

5-43 在图 5-30 中, 在什么情况下会发生从状态 SYN-SENT 到状态 SYN-RCVD 的变迁?

5-44 试以具体例子说明为什么一个运输连接可以有多种方式释放。可以设两个互相通信的用户分别连接在网络的两节点上。

5-45 解释为什么突然释放运输连接就可能会丢失用户数据, 而使用 TCP 的连接释放方法就可保证不丢失数据。

5-46 试用具体例子说明为什么在运输连接建立时要使用三报文握手。说明如不这样做可能会出现什么情况。

5-47 一客户向服务器请求建立 TCP 连接。客户在 TCP 连接建立的三报文握手中的最后一个报文段中捎带上一些数据, 请求服务器发送一个长度为 L 字节的文件。假定:

- (1) 客户和服务器之间的数据传送速率是 R 字节/秒，客户与服务器之间的往返时间是 RTT (固定值)。
- (2) 服务器发送的 TCP 报文段的长度都是 M 字节，而发送窗口大小是 nM 字节。
- (3) 所有传送的报文段都不会出现差错 (无重传)，客户收到服务器发来的报文段后就及时发送确认。
- (4) 所有的协议首部开销都可忽略，所有确认报文段和连接建立阶段的报文段的长度都可忽略 (即忽略这些报文段的发送时间)。

试证明，从客户开始发起连接建立到接收服务器发送的整个文件所需的时间 T 是：

$$T = 2 RTT + L/R \quad \text{当 } nM > R(RTT) + M$$

$$\text{或 } T = 2 RTT + L/R + (K-1)[M/R + RTT - nM/R] \quad \text{当 } nM < R(RTT) + M$$

其中， $K = \lceil L/nM \rceil$ ，符号 $\lceil x \rceil$ 表示若 x 不是整数，则把 x 的整数部分加 1。

(提示：求证的第一个等式发生在发送窗口较大的情况，可以连续把文件发送完。求证的第二个等式发生在发送窗口较小的情况，发送几个报文段后就必须停顿下来，等收到确认后再继续发送。建议先画出双方交互的时间图，然后再进行推导。)

- 5-48** 网络允许的最大报文段长度为 128 字节，序号用 8 位表示，报文段在网络中的寿命为 30 秒。求发送报文段的一方所能达到的最高数据率。
- 5-49** 下面是以十六进制格式存储的一个 UDP 首部：

CB84000D001C001C

试问：

- (1) 源端口号是什么？
 - (2) 目的端口号是什么？
 - (3) 这个用户数据报的总长度是多少？
 - (4) 数据长度是多少？
 - (5) 这个分组是从客户到服务器方向的，还是从服务器到客户方向的？
 - (6) 客户进程是什么？
- 5-50** 把图 5-6 计算 UDP 检验和的例子自己具体演算一下，看是否能够得出书上的计算结果。
- 5-51** 在以下几种情况下，UDP 的检验和在发送时的数值分别是多少？
- (1) 发送方决定不使用检验和。
 - (2) 发送方使用检验和，检验和的数值是全 1。
 - (3) 发送方使用检验和，检验和的数值是全 0。
- 5-52** UDP 和 IP 的不可靠程度是否相同？请加以解释。
- 5-53** UDP 用户数据报的最小长度是多少？用最小长度的 UDP 用户数据报构成的最短 IP 数据报的长度是多少？
- 5-54** 某客户使用 UDP 将数据发送给一服务器，数据共 16 字节。试计算在运输层的传输效率 (有用字节与总字节之比)。
- 5-55** 重做习题 5-54，但在 IP 层计算传输效率。假定 IP 首部无选项。
- 5-56** 重做习题 5-54，但在数据链路层计算传输效率。假定 IP 首部无选项，在数据链路层使用以太网。

- 5-57 某客户有 67000 字节的分组。试说明怎样使用 UDP 数据报将这个分组进行传送。
- 5-58 TCP 在时间为 4:30:20 (即 4 点 30 分 20 秒) 发送了一个报文段。由于没有收到确认, 因此在 4:30:25 重传了前面这个报文段, 并在 4:30:27 收到了确认。若以前的 RTT 值是 4 秒, 根据 Karn 算法, 新的 RTT 值是多少?
- 5-59 TCP 连接使用 1000 字节的窗口值, 而上一次的确认号是 22001。现在收到了一个报文段, 确认号是 22401。试用图来说明在这之前与之后的窗口情况。
- 5-60 同上题。但发送方收到确认号是 22401 的报文段时, 其窗口字段变为 1200 字节。试用图来说明在这之前与之后的窗口情况。
- 5-61 在本题中列出的 8 种情况下, 画出发送窗口的变化, 并标明可用窗口的位置。已知主机 A 要向主机 B 发送 3 KB 的数据。在 TCP 连接建立后, A 的发送窗口大小是 2 KB。A 的初始序号是 0。
- (1) 一开始 A 发送 1 KB 的数据。
 - (2) 接着 A 就一直发送数据, 直到把发送窗口用完。
 - (3) 发送方 A 收到对第 1000 号字节的确认报文段。
 - (4) 发送方 A 再发送 850 B 的数据。
 - (5) 发送方 A 收到 $\text{ack} = 900$ 的确认报文段。
 - (6) 发送方 A 收到对第 2047 号字节的确认报文段。
 - (7) 发送方 A 把剩下的数据全部都发送完。
 - (8) 发送方 A 收到 $\text{ack} = 3072$ 的确认报文段。
- 5-62 TCP 连接处于 ESTABLISHED 状态。以下的事件相继发生:
- (1) 收到一个 FIN 报文段。
 - (2) 应用程序发送“关闭”报文。
- 在每一个事件之后, 连接的状态是什么? 在每一个事件之后发生的动作是什么?
- 5-63 TCP 连接处于 SYN-RCVD 状态。以下的事件相继发生:
- (1) 应用程序发送“关闭”报文。
 - (2) 收到 FIN 报文段。
- 在以上的每一个事件之后, 连接的状态是什么? 在每一个事件之后发生的动作是什么?
- 5-64 TCP 连接处于 FIN-WAIT-1 状态。以下的事件相继发生:
- 收到 ACK 报文段。
- 收到 FIN 报文段。
- 发生了超时。
- 在以上的每一个事件之后, 连接的状态是什么? 在每一个事件之后发生的动作是什么?
- 5-65 假定主机 A 向 B 发送一个 TCP 报文段。在这个报文段中, 序号是 50, 而数据一共有 6 字节长。试问, 在这个报文段中的确认字段是否应当写入 56?
- 5-66 主机 A 通过 TCP 连接向 B 发送一个很长的文件, 因此这需要分成很多个报文段来发送。假定某一个 TCP 报文段的序号是 x , 那么下一个报文段的序号是否就是 $x+1$ 呢?
- 5-67 TCP 的吞吐量应当是每秒发送的数据字节数, 还是每秒发送的首部和数据之和的字节数? 吞吐量应当是每秒发送的字节数, 还是每秒发送的比特数?
- 5-68 在 TCP 的连接建立的三报文握手过程中, 为什么第三个报文段不需要对方的确认?

这会不会出现问题？

- 5-69** 现在假定使用类似 TCP 的协议（即使用滑动窗口可靠传送字节流），数据传输速率是 1 Gbit/s，而网络的往返时间 $RTT = 140\text{ ms}$ 。假定报文段的最大生存时间是 60 秒。如果要尽可能快地传送数据，在我们的通信协议的首部中，发送窗口和序号字段至少各应当设为多大？
- 5-70** 假定用 TCP 协议在 40 Gbit/s 的线路上传送数据。
- (1) 如果 TCP 充分利用了线路的带宽，那么需要多长的时间 TCP 会发生序号绕回？
 - (2) 假定现在 TCP 的首部中采用了时间戳选项。时间戳占用了 4 字节，共 32 位。每隔一定的时间（这段时间叫作一个嘀嗒）时间戳的数值加 1。假定设计的时间戳是每隔 859 微秒，时间戳的数值加 1。试问要经过多少时间才发生时间戳数值的绕回？
- 5-71** 在 5.5 节中指出：例如，若用 2.5 Gbit/s 的速率发送报文段，则不到 14 秒钟序号就会重复。请计算验证这句话。
- 5-72** 已知 TCP 的接收窗口大小是 600（单位是字节，为简单起见以后就省略了单位），已经确认了的序号是 300。试问，在不断地接收报文段和发送确认报文段的过程中，接收窗口也可能会发生变化（增大或缩小）。请用具体例子（指出接收方发送的确认报文段中的重要信息）来说明哪些情况是可能发生的，而哪些情况是不允许发生的。
- 5-73** 在上题中，如果接收方突然因某种原因不能够再接收数据了，可以立即向发送方发送把接收窗口置为零的报文段（即 $rwnd = 0$ ）。这时会导致接收窗口的前沿后退。试问这种情况是否允许？
- 5-74** 流量控制和拥塞控制的最主要的区别是什么？发送窗口的大小取决于流量控制还是拥塞控制？

第6章 应用层

在前五章我们已经详细地讨论了计算机网络提供通信服务的过程。但是我们还没有讨论这些通信服务是如何提供给应用进程来使用的。本章讨论各种应用进程通过什么样的应用层协议来使用网络所提供的这些通信服务。

在上一章，我们已学习了运输层为应用进程提供了端到端的通信服务。但不同的网络应用的应用进程之间，还需要有不同的通信规则。因此在运输层协议之上，还需要有**应用层协议(application layer protocol)**。这是因为，每个应用层协议都是为了解决某一类应用问题，而问题的解决又必须通过位于不同主机中的多个应用进程之间的通信和协同工作来完成。应用进程之间的这种通信必须遵循严格的规则。应用层的具体内容就是精确定义这些通信规则。具体来说，应用层协议应当定义：

- 应用进程交换的报文类型，如请求报文和响应报文。
- 各种报文类型的语法，如报文中的各个字段及其详细描述。
- 字段的语义，即包含在字段中的信息的含义。
- 进程何时、如何发送报文，以及对报文进行响应的规则。

互联网公共领域的标准应用的应用层协议是由 RFC 文档定义的，大家都可以使用。例如，万维网的应用层协议 HTTP（超文本传输协议）就是由建议标准 RFC 7230 定义的。如果浏览器开发者遵守 RFC 7230 标准，所开发出来的浏览器就能够访问任何遵守该标准的万维网服务器并获取相应的万维网页面。在互联网中还有很多其他应用的应用层协议不是公开的，而是专用的。例如，很多现有的 P2P 文件共享系统使用的就是专用应用层协议。

请注意，应用层协议与网络应用并不是同一个概念。应用层协议只是网络应用的一部分。例如，万维网应用是一种基于客户/服务器体系结构的网络应用。万维网应用包含很多部件，有万维网浏览器、万维网服务器、万维网文档的格式标准，以及一个应用层协议。万维网的应用层协议是 HTTP，它定义了万维网浏览器和万维网服务器之间传送的报文类型、格式和序列等规则。而万维网浏览器如何显示一个万维网页面，万维网服务器是用多线程还是用多进程来实现，则都不是 HTTP 所定义的内容。

应用层的许多协议都是基于**客户服务器方式**。即使是 P2P 对等通信方式，实质上也是一种特殊的客户服务器方式。这里再明确一下，**客户(client)**和**服务器(server)**都是指通信中所涉及的两个**应用进程**。客户服务器方式所描述的是进程之间服务和被服务的关系。这里最主要的特征就是：**客户是服务请求方，服务器是服务提供方**。

下面先讨论许多应用协议都要使用的域名系统。在介绍了文件传送协议和远程登录协议后，再重点介绍万维网的工作原理及其主要协议。由于万维网的出现使互联网得到了飞速的发展，因此万维网在本章中占有最大的篇幅，也是本章的重点。接着讨论用户经常使用的电子邮件。最后，介绍有关网络管理方面的问题以及有关网络编程的基本概念。对应用层更深入的学习可参阅[COME15][COME06][KUROI17][TANE11]及有关标准。

本章最重要的内容是：

(1) 域名系统 DNS——从域名解析出 IP 地址。

- (2) 万维网和 HTTP 协议，以及万维网的两种不同的信息搜索引擎。
- (3) 电子邮件的传送过程，SMTP 协议和 POP3 协议、IMAP 协议使用的场合。
- (4) 动态主机配置协议 DHCP 的特点。
- (5) 网络管理的三个组成部分（SNMP 本身、管理信息结构 SMI 和管理信息库 MIB）的作用。
- (6) 系统调用和应用编程接口的基本概念。
- (7) P2P 文件系统。

6.1 域名系统 DNS

6.1.1 域名系统概述

域名系统 DNS (Domain Name System) 是互联网使用的命名系统，用来把便于人们使用的机器名字转换为 IP 地址。域名系统其实就是名字系统。为什么不叫“名字”而叫“域名”呢？这是因为在这种互联网的命名系统中使用了许多“域”(domain)，因此就出现了“域名”这个名词。“域名系统”很明确地指明这种系统是用在互联网中的。

许多应用层软件经常直接使用域名系统 DNS。虽然计算机的用户只是间接而不是直接使用域名系统，但 DNS 却为互联网的各种网络应用提供了核心服务。

用户与互联网上某台主机通信时，必须要知道对方的 IP 地址。然而用户很难记住长达 32 位的二进制主机地址。即使是点分十进制 IP 地址也并不太容易记忆。但在应用层为了便于用户记忆各种网络应用，连接在互联网上的主机不仅有 IP 地址，而且还有便于用户记忆的主机名字。域名系统 DNS 能够把互联网上的主机名字转换为 IP 地址。

早在 ARPANET 时代，整个网络上只有数百台计算机，那时使用一个叫作 hosts 的文件，列出所有主机名字和相应的 IP 地址。只要用户输入一台主机名字，计算机就可很快地把这台主机名字转换成机器能够识别的二进制 IP 地址。

为什么机器在处理 IP 数据报时要使用 IP 地址而不使用域名呢？这是因为 IP 地址的长度是固定的 32 位（如果是 IPv6 地址，那就是 128 位，也是定长的），而域名的长度并不是固定的，机器处理起来比较困难。

从理论上讲，整个互联网可以只使用一个域名服务器，使它装入互联网上所有的主机名，并回答所有对 IP 地址的查询。然而这种做法并不可取。因为互联网规模很大，这样的域名服务器肯定会因过负荷而无法正常工作，而且一旦域名服务器出现故障，整个互联网就会瘫痪。因此，早在 1983 年互联网就开始采用层次树状结构的命名方法，并使用分布式的域名系统 DNS [RFC 1034, RFC 1035, STD13]。

互联网的域名系统 DNS 被设计成为一个联机分布式数据库系统，并采用客户服务器方式。DNS 使大多数名字都在本地进行解析(resolve)^①，仅少量解析需要在互联网上通信，因此 DNS 系统的效率很高。由于 DNS 是分布式系统，即使单个计算机出了故障，也不会妨碍整个 DNS 系统的正常运行。

^① 注：在 TCP/IP 的文档中，这种地址转换常称为地址解析。解析就是转换的意思，地址解析可能会包含多次的查询请求和回答过程。

域名到 IP 地址的解析是由分布在互联网上的许多**域名服务器程序**（可简称为域名服务器）共同完成的。域名服务器程序在专设的节点上运行，而人们也常把运行域名服务器程序的机器称为**域名服务器**。

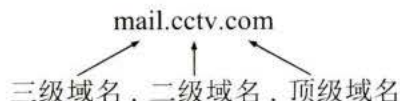
域名到 IP 地址的解析过程的要点如下：当某一个应用进程需要把主机名解析为 IP 地址时，该应用进程就调用**解析程序(resolver)**，并成为 DNS 的一个客户，把待解析的域名放在 DNS 请求报文中，以 UDP 用户数据报方式发给本地域名服务器（使用 UDP 是为了减少开销）。本地域名服务器在查找域名后，把对应的 IP 地址放在回答报文中返回。应用进程获得目的主机的 IP 地址后即可进行通信。

若本地域名服务器不能回答该请求，则此域名服务器就暂时成为 DNS 中的另一个客户，并向其他域名服务器发出查询请求。这种过程直至找到能够回答该请求的域名服务器为止。上述这种查找过程，后面还要进一步讨论。

6.1.2 互联网的域名结构

早期的互联网使用了非等级的名字空间，其优点是名字简短。但当互联网上的用户数急剧增加时，用非等级的名字空间来管理一个很大的而且是经常变化的名字集合是非常困难的。因此，互联网后来就采用了层次树状结构的命名方法，就像全球邮政系统和电话系统那样。采用这种命名方法，任何一个连接在互联网上的主机或路由器，都有一个唯一的**层次结构的**名字，即**域名(domain name)**。这里，“域”(domain)是名字空间中一个可被管理的划分。域还可以划分为子域，而子域还可继续划分为子域的子域，这样就形成了顶级域、二级域、三级域，等等。

从语法上讲，每一个域名都由**标号(label)**序列组成，而各标号之间用点隔开（请注意，这里所说的“点”是英语中的句号“.”，不是中文的句号“。”）。例如下面的域名



就是中央电视台用于收发电子邮件的计算机（即邮件服务器）的域名，它由三个标号组成，其中标号 com 是顶级域名，标号 cctv 是二级域名，标号 mail 是三级域名。

DNS 规定，域名中的标号都由英文字母和数字组成，**每一个标号不超过 63 个字符**（但为了记忆方便，最好不要超过 12 个字符），**也不区分大小写字母**（例如，CCTV 或 cctv 在域名中是等效的）。标号中除连字符(-)外不能使用其他的标点符号。级别最低的域名写在最左边，而级别最高的顶级域名则写在最右边。**由多个标号组成的完整域名总共不超过 255 个字符**。DNS 既不规定一个域名需要包含多少个下级域名，也不规定每一级的域名代表什么意思。各级域名由其上一级的域名管理机构管理，而最高的顶级域名则由 ICANN 进行管理。用这种方法可使每一个域名在整个互联网范围内是唯一的，并且也容易设计出一种查找域名的机制。

需要注意的是，域名只是个**逻辑概念**，并不代表计算机所在的物理地点。变长的域名和使用有助记忆的字符串，是为了便于人使用。而 IP 地址是定长的 32 位二进制数字则非常便于机器进行处理。这里需要注意，域名中的“点”和点分十进制 IP 地址中的“点”并无一一对应的关系。点分十进制 IP 地址中一定是包含三个“点”，但每一个域名中“点”的数

目则不一定正好是三个。

截至 2020 年 6 月的统计[W-Wiki]，现在全球顶级域名 TLD (Top Level Domain)在 IANA 登记的已有 1584 个（其中有不到 80 个未使用）。原先的顶级域名共分为三大类：

(1) **国家顶级域名 nTLD**：采用 ISO 3166 的规定。如：cn 表示中国，us 表示美国，uk 表示英国，等等^①。国家顶级域名又常记为 ccTLD（cc 表示国家代码 country-code）。截至 2020 年 6 月为止，国家顶级域名总数已达 316 个。

(2) **通用顶级域名 gTLD**：到 2006 年 12 月为止，通用顶级域名的总数已经达到 20 个。最先确定的通用顶级域名有 7 个，即：

com（公司企业），net（网络服务机构），org（非营利性组织），int（国际组织），edu（美国专用的教育机构），gov（美国的政府部门），mil 表示（美国的军事部门）。

以后又陆续增加了 13 个通用顶级域名：

aero（航空运输企业），asia（亚太地区），biz（公司和企业），cat（使用加泰隆人的语言和文化团体），coop（合作团体），info（各种情况），jobs（人力资源管理者），mobi（移动产品与服务的用户和提供者），museum（博物馆），name（个人），pro（有证书的专业人员），tel（Telnic 股份有限公司），travel（旅游业）。

(3) **基础结构域名(infrastructure domain)**：这种顶级域名只有一个，即 arpa，用于反向域名解析，因此又称为**反向域名**。

值得注意的是，ICANN 于 2011 年 6 月 20 日在新加坡会议上正式批准**新顶级域名 (New gTLD)**，因此任何公司、机构都有权向 ICANN 申请新的顶级域名。新顶级域名的后缀特点，使企业域名具有了显著的、强烈的标志特征。因此，新顶级域名被认为是真正的企业网络商标。新顶级域名是企业品牌战略发展的重要内容，其申请费很高（18 万美元），并且在 2013 年开始启用。目前已有一些由两个汉字组成的中文的顶级域名出现了，例如，商城、公司、新闻等。然而中文顶级域名并未获得广泛的使用（可能是使用不太方便吧）。

在国家顶级域名下注册的二级域名均由该国家自行确定。例如，顶级域名为 jp 的日本，将其教育和企业机构的二级域名定为 ac 和 co，而不用 edu 和 com。

我国把二级域名划分为“**类别域名**”和“**行政区域名**”两大类。

“类别域名”共 7 个，分别为：ac（科研机构），com（工、商、金融等企业），edu（中国的教育机构），gov（中国的政府机构），mil（中国的国防机构），net（提供互联网络服务的机构），org（非营利性的组织）。

“行政区域名”共 34 个，适用于我国的各省、自治区、直辖市。例如：bj（北京市），js（江苏省），等等。

关于我国的互联网络发展现状以及各种规定（如申请域名的手续），均可在**中国互联网网络信息中心 CNNIC** 的网址上找到[W-CNNIC]。

用域名树来表示互联网的域名系统是最清楚的。图 6-1 是互联网域名空间的结构，它实

^① 注：实际上，国家顶级域名也包括某些地区的域名，如我国的香港特区（hk）和台湾省（tw）也都是 ccTLD 里面的顶级域名。此外，国家顶级域名可以使用一个国家自己的文字。例如，中国可以有“.cn”、“.中国”和繁体字的“.中國”这三种不同形式的域名。

际上是一个倒过来的树，在最上面的是根，但没有对应的名字。根下面一级的节点^①就是最高一级的顶级域名（由于根没有名字，所以在根下面一级的域名就叫作顶级域名）。顶级域名可往下划分子域，即二级域名。再往下划分就是三级域名、四级域名，等等。图 6-1 列举了一些域名作为例子。凡是在顶级域名 `com` 下注册的单位都获得了一个二级域名。图中给出的例子有：中央电视台 `cctv`，以及 `IBM` 和 `华为` 等公司。在顶级域名 `cn`（中国）下面举出了几个二级域名，如：`bj`、`edu` 以及 `com`。在某个二级域名下注册的单位就可以获得一个三级域名。图中给出的在 `edu` 下面的三级域名有：`tsinghua`（清华大学）和 `pku`（北京大学）。一旦某个单位拥有了一个域名，它就可以自己决定是否要进一步划分其下属的子域，并且不必由其上级机构批准。图中 `cctv`（中央电视台）和 `tsinghua`（清华大学）都分别划分了自己的下一级的域名 `mail` 和 `www`（分别是三级域名和四级域名）^②。域名树的树叶就是单台计算机的名字，它不能再继续往下划分子域了。

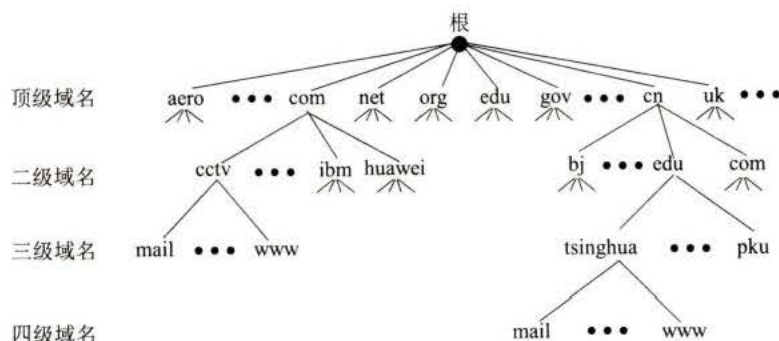


图 6-1 互联网的域名空间

应当注意，虽然中央电视台和清华大学都各有一台计算机取名为 `mail`，但它们的域名并不一样，因为前者是 `mail.cctv.com`，而后者是 `mail.tsinghua.edu.cn`。因此，即使在世界上还有很多单位的计算机取名为 `mail`，但是它们在互联网中的域名都必须是唯一的。

这里还要强调指出，互联网的名字空间是按照机构的组织来划分的，与物理的网络无关，与 IP 地址中的“子网”也没有关系。

6.1.3 域名服务器

上面讲述的域名体系是抽象的。但具体实现域名系统则是使用分布在各地的域名服务器。从理论上讲，可以让每一级的域名都有一个相对应的域名服务器，使所有的域名服务器构成和图 6-1 相对应的“域名服务器树”的结构。但这样做会使域名服务器的数量太多，使域名系统的运行效率降低。因此 DNS 就采用划分区的方法来解决这个问题。

一个服务器所负责管辖的（或有权限的）范围叫作区(zone)。各单位根据具体情况来划分自己管辖范围的区。但在一个区中的所有节点必须是能够连通的。每一个区设置相应的权限域名服务器(authoritative name server)，用来保存该区中的所有主机的域名到 IP 地址的映射。总之，DNS 服务器的管辖范围不是以“域”为单位，而是以“区”为单位的。区是 DNS

① 注：根据[MINGCI94]，对于树这样的数据结构，它的 node 应当译为“节点”（不是结点）。

② 注：为了便于记忆，人们愿意把用作邮件服务器的计算机取名为 `mail`，而把用作网站服务器的计算机取名为 `www`。

服务器实际管辖的范围。区可能等于或小于域，但一定不能大于域。

图 6-2 是区的不同划分方法的举例。假定 abc 公司有下属部门 x 和 y，部门 x 下面又分三个分部门 u、v 和 w，而 y 下面还有其下属部门 t。图 6-2(a)表示 abc 公司只设一个区 abc.com。这时，区 abc.com 和域 abc.com 指的是同一件事。但图 6-2(b)表示 abc 公司划分了两个区（大的公司可能要划分多个区）：abc.com 和 y.abc.com。这两个区都隶属于域 abc.com，都各设置了相应的权限域名服务器。不难看出，区是“域”的子集。

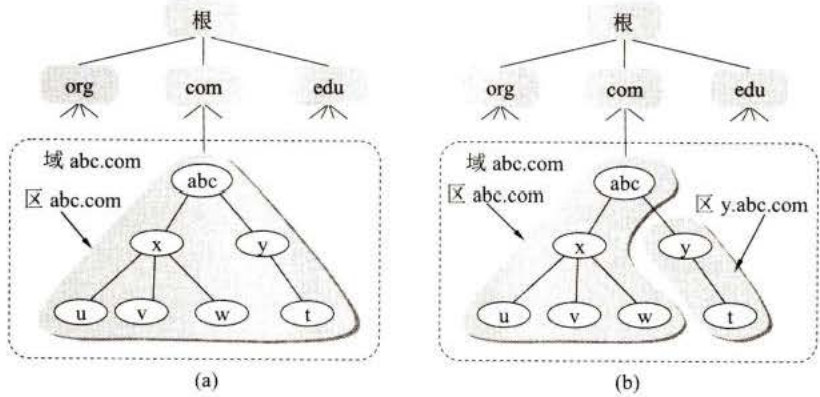


图 6-2 DNS 划分区的举例

图 6-3 以图 6-2(b)中公司 abc 划分的两个区为例，给出了 DNS 域名服务器树状结构图。这种 DNS 域名服务器树状结构图可以更准确地反映出 DNS 的分布式结构。在图 6-3 中的每一个域名服务器都能够进行部分域名到 IP 地址的解析。当某个 DNS 服务器不能进行域名到 IP 地址的转换时，它就设法找互联网上别的域名服务器进行解析。

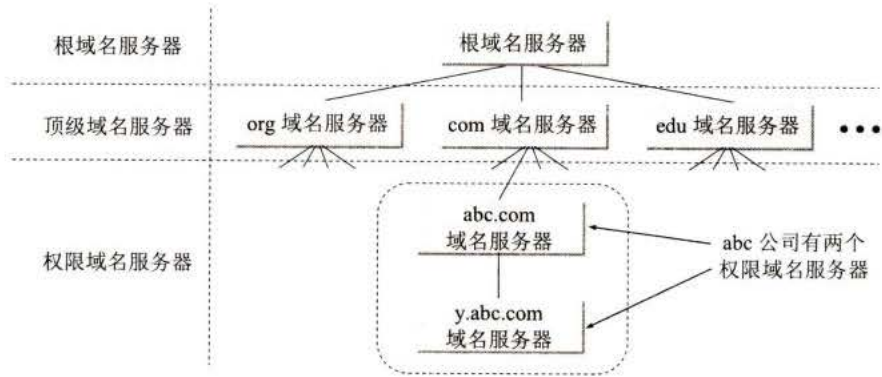


图 6-3 树状结构的 DNS 域名服务器

从图 6-3 可看出，互联网上的 DNS 域名服务器也是按照层次安排的。每一个域名服务器都只对域名体系中的一部分进行管辖。根据域名服务器所起的作用，可以把域名服务器划分为以下四种不同的类型：

(1) **根域名服务器(root name server)**：根域名服务器是最高层次的域名服务器，也是最重要的域名服务器。所有的根域名服务器都知道所有的顶级域名服务器的域名和 IP 地址。根域名服务器是最重要的域名服务器，因为不管是哪一个本地域名服务器，若要对互联网上任何一个域名进行解析（即转换为 IP 地址），只要自己无法解析，就首先求助于根域名服务

器。假定所有的根域名服务器都瘫痪了，那么整个互联网中的 DNS 系统就无法工作。全世界的根域名服务器只使用 13 个不同 IP 地址的域名，即 a.rootservers.net, b.rootservers.net, ..., m.rootservers.net。每个域名下的根域名服务器由专门的公司或美国政府的某个部门负责运营。但请注意，虽然互联网的根域名服务器总共只有 13 个域名，但**根域名服务器并非仅由 13 台机器所组成**（如果仅仅依靠这 13 台机器，根本不可能为全世界的互联网用户提供令人满意的服务）。实际上，在互联网中是由 13 套装置（13 installations，也就是 13 套系统）构成这 13 组根域名服务器的[W-ROOT]。每一套装置在很多地点安装根域名服务器（也可称为镜像根服务器），但都使用同一个域名。负责运营根域名服务器的公司大多在美国，但所有的根域名服务器却分布在全世界。为了提供更可靠的服务，在每一个地点的根域名服务器往往由**多台机器组成**（为了安全起见，这些根域名服务器的具体位置是严格保密的，不对外开放参观）。现在世界上大部分 DNS 域名服务器，都能**就近**找到一个根域名服务器查询 IP 地址（现在这些根域名服务器都已增加了 IPv6 地址）。为了方便，人们常用从 A 到 M 的前 13 个英文字母中的一个，来表示某组根域名服务器。截至 2021 年 3 月 24 日，全球共有 1375 个根域名服务器在运行，其中在我国的共有 37 个（分布在北京(8 个)、上海、杭州(2 个)、武汉(2 个)、贵阳、重庆、广州、西宁(3 个)、郑州(2 个)、香港(7 个)、澳门(2)、台北(7 个)）。

由于根域名服务器采用了**任播(anycast)**技术^①，因此当 DNS 客户向某个根域名服务器的 IP 地址发出查询报文时，互联网上的路由器就能找到离这个 DNS 客户最近的一个根域名服务器。这样做不仅加快了 DNS 的查询过程，也更加合理地利用了互联网的资源。

必须指出，目前根域名服务器在全球的分布仍然是很不均衡的。在某些地区根域名服务器还较少，这就影响了上网的速率。

需要注意的是，在许多情况下，根域名服务器并不直接把待查询的域名直接转换成 IP 地址（根域名服务器也没有存放这种信息），而是告诉本地域名服务器下一步应当找哪一个顶级域名服务器进行查询。

由于根域名服务器在 DNS 中的地位特殊，因此对根域名服务器有许多具体的要求，如必须能够运行某些程序等[RFC 7720]。

(2) **顶级域名服务器**（即 TLD 服务器）：这些域名服务器负责管理在该顶级域名服务器注册的所有二级域名。当收到 DNS 查询请求时，就给出相应的回答（可能是最后的结果，也可能是下一步应当找的域名服务器的 IP 地址）。

(3) **权限域名服务器**：这就是前面已经讲过的负责一个区的域名服务器。当一个权限域名服务器还不能给出最后的查询回答时，就会告诉发出查询请求的 DNS 客户，下一步应当找哪一个权限域名服务器。例如在图 6-2(b)中，区 abc.com 和区 y.abc.com 各设有一个权限域名服务器。

(4) **本地域名服务器(local name server)**：本地域名服务器并不属于图 6-3 所示的域名服务器层次结构，但它对域名系统非常重要。当一台主机发出 DNS 查询请求时，这个查询请求报文就发送给本地域名服务器。由此可看出本地域名服务器的重要性。每一个互联网服务提供者 ISP，或一个大学，甚至一个大学里的系，都拥有一个**本地域名服务器**，本地域名服务器离用户较近，一般不超过几个路由器的距离。当所要查询的主机也属于同一个本地 ISP

① 注：任播的 IP 数据报的终点是一组在不同地点的主机，但具有相同的 IP 地址。IP 数据报交付离源点最近的一台主机。

时,该本地域名服务器立即就能将所查询的主机名转换为它的 IP 地址,而不需要再去询问其他的域名服务器。

为了提高域名服务器的可靠性,DNS 域名服务器都把数据复制到几个域名服务器来保存,其中的一个是主域名服务器(master name server),其他的是辅助域名服务器(secondary name server)。当主域名服务器出故障时,辅助域名服务器可以保证 DNS 的查询工作不会中断。主域名服务器定期把数据复制到辅助域名服务器中,而更改数据只能在主域名服务器中进行。这样就保证了数据的一致性。

下面简单讨论一下域名的解析过程。这里要注意两点。

第一,主机向本地域名服务器的查询一般都采用递归查询(recursive query)。所谓递归查询就是:如果主机所询问的本地域名服务器不知道被查询域名的 IP 地址,那么本地域名服务器就以 DNS 客户的身份,向其他根域名服务器继续发出查询请求报文(即替该主机继续查询),而不是让该主机自己进行下一步的查询。因此,递归查询返回的查询结果或者是所要查询的 IP 地址,或者是报错,表示无法查询到所需的 IP 地址。

第二,本地域名服务器向根域名服务器的查询通常采用迭代查询(iterative query)。迭代查询的特点是这样的:当根域名服务器收到本地域名服务器发出的迭代查询请求报文时,要么给出所要查询的 IP 地址,要么告诉本地域名服务器:“你下一步应当向哪一个域名服务器进行查询”。然后让本地域名服务器进行后续的查询(而不是替本地域名服务器进行后续的查询)。根域名服务器通常是把自己知道的顶级域名服务器的 IP 地址告诉本地域名服务器,让本地域名服务器再向顶级域名服务器查询。顶级域名服务器在收到本地域名服务器的查询请求后,要么给出所要查询的 IP 地址,要么告诉本地域名服务器下一步应当向哪一个权限域名服务器进行查询,本地域名服务器就这样进行迭代查询。最后,知道了所要解析的域名的 IP 地址,然后把这个结果返回给发起查询的主机。当然,本地域名服务器也可以采用递归查询,这取决于最初的查询请求报文的设置要求使用哪一种查询方式。

图 6-4 用例子说明了这两种查询的区别。

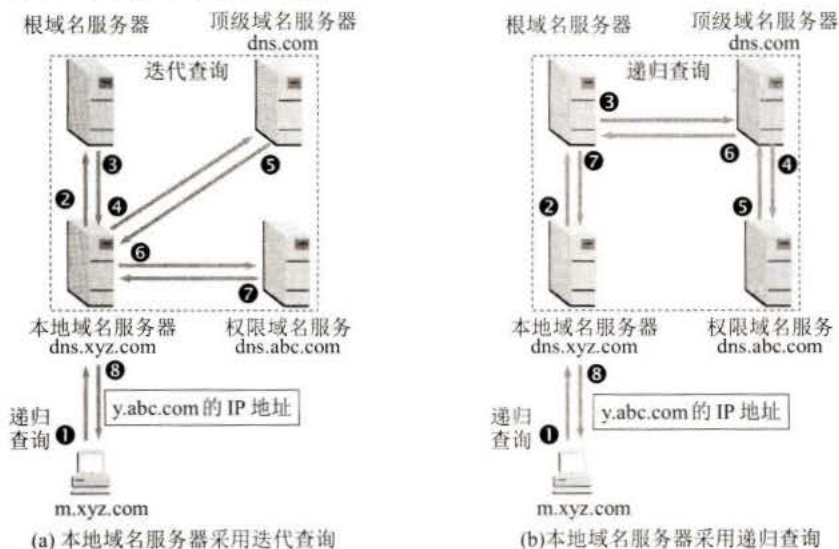


图 6-4 DNS 查询举例

假定域名为 m.xyz.com 的主机想知道另一台主机(域名为 y.abc.com)的 IP 地址。例如,

主机 `m.xyz.com` 打算发送邮件给主机 `y.abc.com`。这时就必须知道主机 `y.abc.com` 的 IP 地址。下面是图 6-4(a)的几个查询步骤：

- ❶ 主机 `m.xyz.com` 先向其本地域名服务器 `dns.xyz.com` 进行递归查询。
- ❷ 本地域名服务器采用迭代查询。它先向一个根域名服务器查询。
- ❸ 根域名服务器告诉本地域名服务器，下一次应查询的顶级域名服务器 `dns.com` 的 IP 地址。
- ❹ 本地域名服务器向顶级域名服务器 `dns.com` 进行查询。
- ❺ 顶级域名服务器 `dns.com` 告诉本地域名服务器，下一次应查询的权限域名服务器 `dns.abc.com` 的 IP 地址。
- ❻ 本地域名服务器向权限域名服务器 `dns.abc.com` 进行查询。
- ❼ 权限域名服务器 `dns.abc.com` 告诉本地域名服务器，所查询的主机的 IP 地址。
- ❽ 本地域名服务器最后把查询结果告诉主机 `m.xyz.com`。

我们注意到，这 8 个步骤总共要使用 8 个 UDP 用户数据报的报文。本地域名服务器经过三次迭代查询后，从权限域名服务器 `dns.abc.com` 得到了主机 `y.abc.com` 的 IP 地址，最后把结果返回给发起查询的主机 `m.xyz.com`。

图 6-4(b)是本地域名服务器采用递归查询的情况。在这种情况下，本地域名服务器只需向根域名服务器查询一次，后面的几次查询都是在其他几个域名服务器之间进行的（步骤❸至❻）。只是在步骤❼，本地域名服务器从根域名服务器得到了所需的 IP 地址。最后在步骤❽，本地域名服务器把查询结果告诉主机 `m.xyz.com`。整个的查询也是使用 8 个 UDP 报文。

为了提高 DNS 查询效率，并减轻根域名服务器的负荷和减少互联网上的 DNS 查询报文数量，在域名服务器中广泛地使用了**高速缓存**（有时也称为**高速缓存域名服务器**）。高速缓存用来存放最近查询过的域名以及从何处获得域名映射^①信息的记录。

例如，在图 6-4(a)的查询过程中，如果在不久前已经有用户查询过域名为 `y.abc.com` 的 IP 地址，那么本地域名服务器就不必向根域名服务器重新查询 `y.abc.com` 的 IP 地址，而是直接把高速缓存中存放的上次查询结果（即 `y.abc.com` 的 IP 地址）告诉用户。

假定本地域名服务器的缓存中并没有 `y.abc.com` 的 IP 地址，而是存放着顶级域名服务器 `dns.com` 的 IP 地址，那么本地域名服务器也可以不向根域名服务器进行查询，而是直接向 `com` 顶级域名服务器发送查询请求报文。这样不仅可以大大减轻根域名服务器的负荷，而且也能够使互联网上的 DNS 查询请求和回答报文的数量大为减少。

由于名字到地址的绑定^②并不经常改变，为保持高速缓存中的内容正确，域名服务器应为每项内容设置计时器并处理超过合理时间的项目（例如，每个项目只存放两天）。当域名服务器已从缓存中删去某项信息后又被请求查询该项信息，就必须重新到授权管理该项的域名服务器中获取绑定信息。当权限域名服务器回答一个查询请求时，在响应中都指明绑定有效存在的时间值。增加此时间值可减少网络开销，而减少此时间值可提高域名转换的准确性。

不但在本地域名服务器中需要高速缓存，在主机中也很需要。许多主机在启动时从本

① 注：映射(mapping)指两个集合元素之间的一种对应规则。

② 注：绑定(binding)指一个对象（或事务）与其某种属性建立某种联系的过程。

地域名服务器下载名字和地址的全部数据库，维护存放自己最近使用的域名的高速缓存，并且只在从缓存中找不到名字时才使用域名服务器。维护本地域名服务器数据库的主机自然应该定期地检查域名服务器以获取新的映射信息，而且主机必须从缓存中删掉无效的项。由于域名改动并不频繁，大多数网点不需花太多精力就能维护数据库的一致性。

6.2 文件传送协议

6.2.1 FTP 概述

文件传送协议 FTP (File Transfer Protocol) [RFC 959, STD9]曾是互联网上使用得最广泛的文件传送协议。FTP 提供交互式的访问，允许客户指明文件的类型与格式（如指明是否使用 ASCII 码），并允许文件具有存取权限（如访问文件的用户必须经过授权，并输入有效的口令）。FTP 屏蔽了各计算机系统的细节，因而适合于在异构网络中任意计算机之间传送文件。

在互联网发展的早期阶段，用 FTP 传送文件约占整个互联网的通信量的三分之一，而由电子邮件和域名系统所产生的通信量远小于 FTP 所产生的通信量。只是到了 1995 年，WWW 的通信量才首次超过了 FTP。

在下面 6.2.2 和 6.2.3 节分别介绍基于 TCP 的 FTP 和基于 UDP 的简单文件传送协议 TFTP，它们都是文件共享协议中的一大类，即**复制整个文件**，其特点是：若要存取一个文件，就必须先获得一个本地的文件副本。如果要修改文件，只能对文件的副本进行修改，然后再将修改后的文件副本传回到原节点。

文件共享协议中的另一大类是**联机访问(on-line access)**。联机访问意味着允许多个程序同时对一个文件进行存取。和数据库系统的不同之处是用户不需要调用一个特殊的客户进程，而是由操作系统提供对远地共享文件进行访问的服务，就如同对本地文件的访问一样。这就使用户可以用远地文件作为输入和输出来运行任何应用程序，而操作系统中的文件系统则提供对共享文件的**透明存取**。透明存取的优点是：将原来用于处理本地文件的应用程序用来处理远地文件时，不需要对该应用程序做明显的改动。属于文件共享协议的有**网络文件系统 NFS (Network File System) [COME06]**。网络文件系统 NFS 最初是在 UNIX 操作系统环境下实现文件和目录共享的。NFS 可使本地计算机共享远地的资源，就像这些资源在本地一样。由于 NFS 原先是美国 SUN 公司在 TCP/IP 网络上创建的，因此目前 NFS 主要应用在 TCP/IP 网络上。然而现在 NFS 也可在 OS/2, MS-Windows, NetWare 等操作系统上运行。NFS 还没有成为互联网的正式标准。经过几次修订更新，现在的最新版本（NFSv4）是 2015 年 3 月发布的建议标准[RFC 7530]。限于篇幅，本书不讨论 NFS 的详细工作过程。

6.2.2 FTP 的基本工作原理

网络环境中的一项基本应用就是将文件从一台计算机中复制到另一台可能相距很远的计算机中。初看起来，在两台主机之间传送文件是很简单的事情。其实这往往非常困难。原因是众多的计算机厂商研制出的文件系统多达数百种，且差别很大。经常遇到的问题是：

- (1) 计算机存储数据的格式不同。
- (2) 文件的目录结构和文件命名的规定不同。
- (3) 对于相同的文件存取功能，操作系统使用的命令不同。

扫一扫



视频讲解

扫一扫



视频讲解